



Machine Learning Project Report

Guest Satisfaction Prediction

Prof: Dr. Dina Khattab
Team ID: SC_6

Level	Department	ID	Name	
3	Scientific Computing (SC)	2022170006	ابراهيم محمد ابراهيم احمد محمد	1
		2022170445	منه حسن محمد حسن	2
		2022170185	سعاد سعيد ابراهيم محمد	3
		2022170569	رغد سامي زكي جوهر عسكر	4
		2022170302	فاطمة الزهراء عاطف محمود	5
		2022170556	حسام الدين احمد رزق خميس	6

2025 – 2026

(Milestone 1)

1. Introduction

This report details the machine learning pipeline implemented by our team to predict guest satisfaction using the dataset `GuestSatisfactionPrediction.csv`. The pipeline includes data loading, exploration, cleaning, preprocessing, feature engineering, and encoding, preparing the data for model training.

2. Data Loading and Initial Exploration

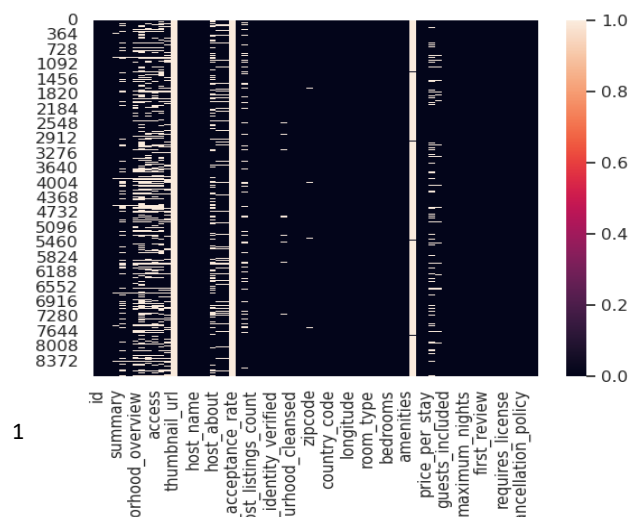
- **Step:** We read the dataset from `GuestSatisfactionPrediction.csv` using pandas and displayed basic details.
- **Details:**
 - Dataset dimensions: 8,724 rows and 69 columns.
 - Key columns include `id`, `listing_url`, `name`, `summary`, `host_response_time`, `room_type`, and `review_scores_rating`.
 - Methods used: `data.shape`, `data.head()`, `data.columns`, `data.describe()`, and `data.info()` to inspect structure, data types, and statistics.
- **Purpose:** Understand the dataset's composition and identify initial preprocessing needs.

3. Checking Duplicates

- **Step:** We checked for duplicate rows using `data.duplicated().sum()`.
- **Details:** No duplicates were found (sum = 0).
- **Purpose:** Ensure data integrity by confirming no redundant entries.

4. Handling Missing Values

- **Step:** We identified null values with `data.isnull().sum()` and calculated the percentage of missing values for each column. A heatmap was drawn to visualize missing data (space reserved below).



- **Details:**

Columns with >80% missing values were dropped: `thumbnail_url`, `host_acceptance_rate`, `square_feet`.

Missing percentages guided further preprocessing decisions.

- **Purpose:** Assess and mitigate the impact of missing data.

5. Data Cleaning

- **Step:** We dropped redundant or irrelevant columns:
 - `name`, `host_name`, `id`, `listing_url`, `host_url`, `host_id`, `country_code` (same as `country`), `smart_location` (same as `city` + `state`), `street` (same as `city` + `state` + `country`).
- **Details:** These columns were removed to simplify the dataset and focus on predictive features.
- **Step:** Cleaned specific columns:
 - Removed "\$" from `nightly_price`, `price_per_stay`, `security_deposit`, `cleaning_fee`, `extra_people`.
 - Removed "%" from `host_response_rate`.
 - Converted `first_review`, `last_review`, `host_since` to days from the current date.
 - Removed "-" from `zipcode`.
- **Purpose:** Standardize data formats for numerical analysis.

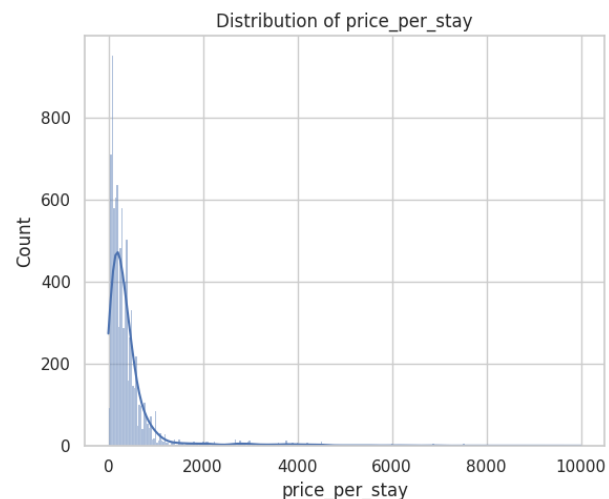
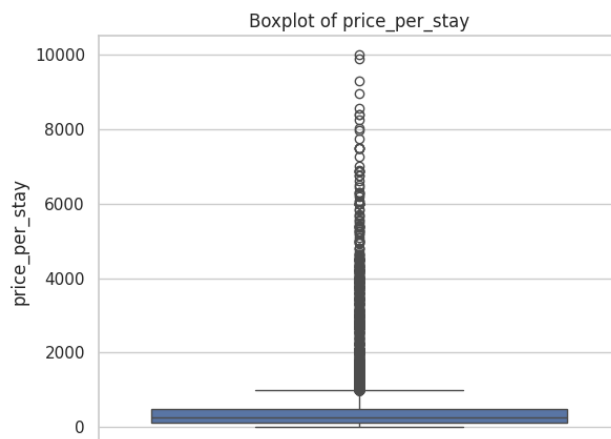
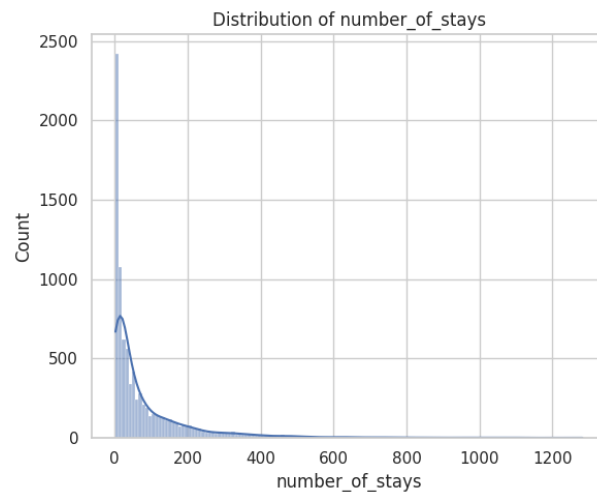
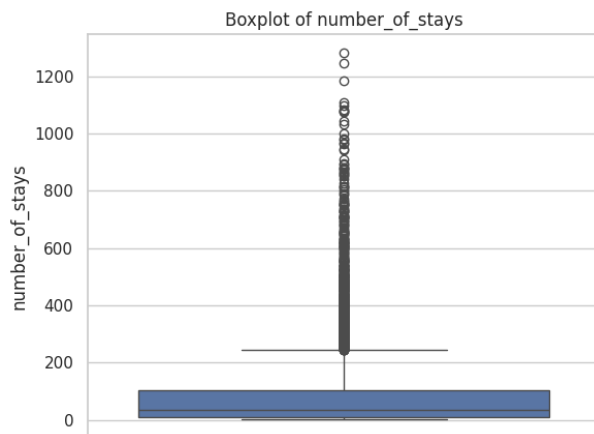
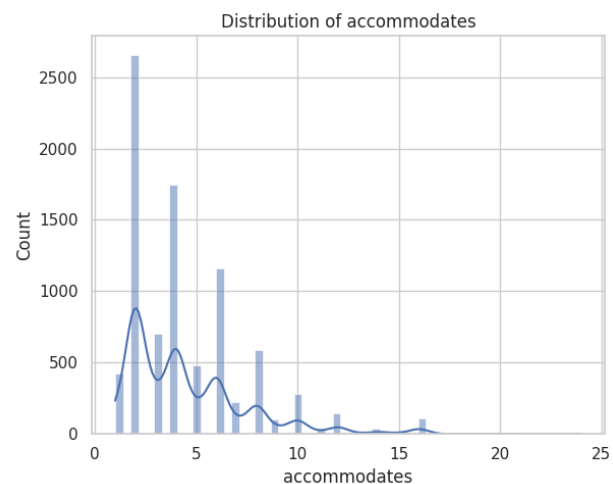
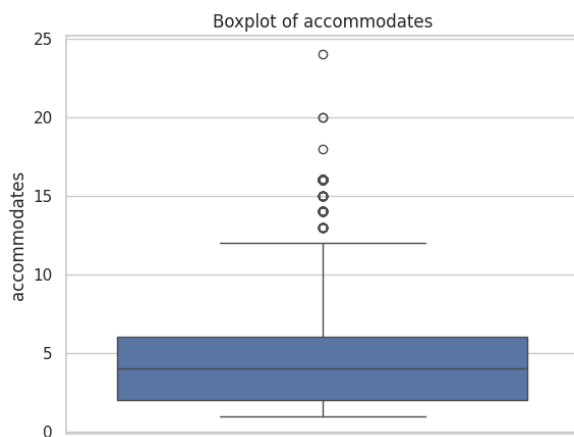
6. Imputing Missing Values

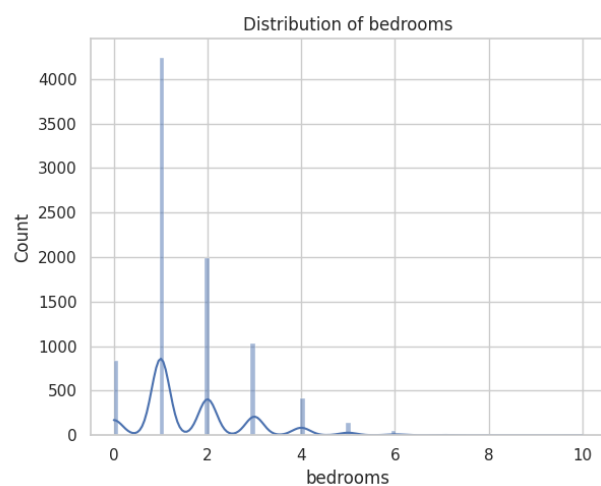
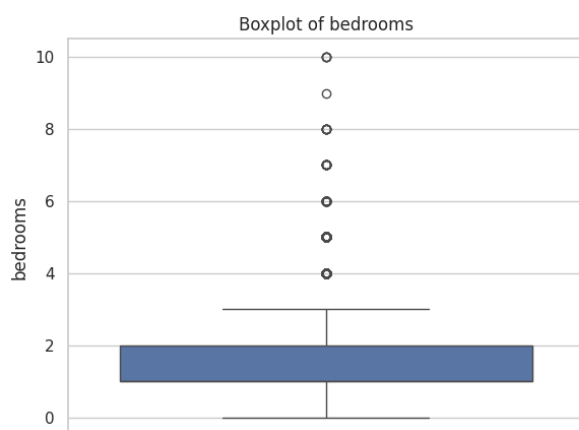
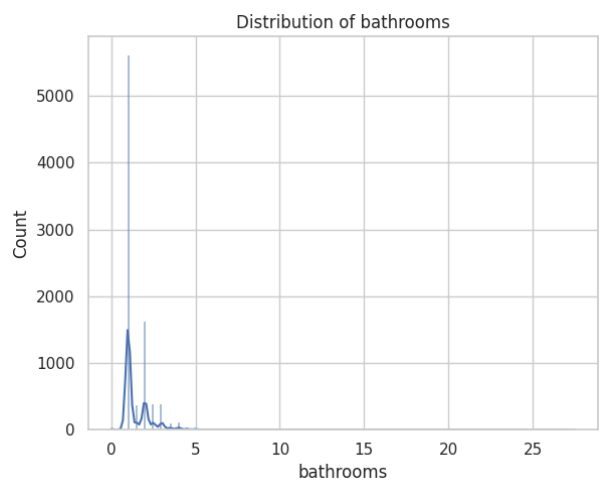
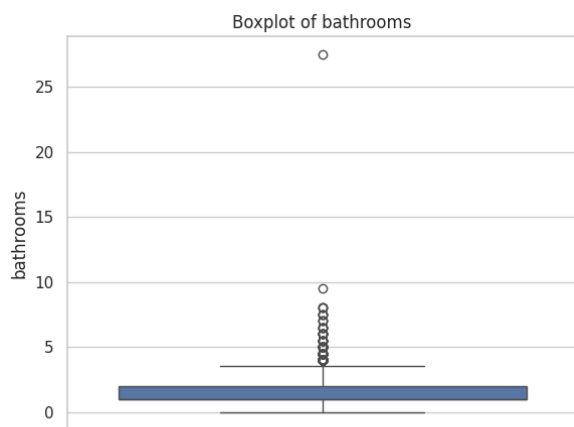
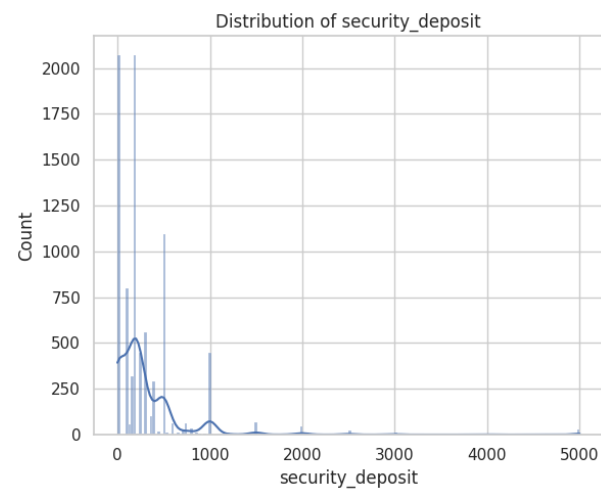
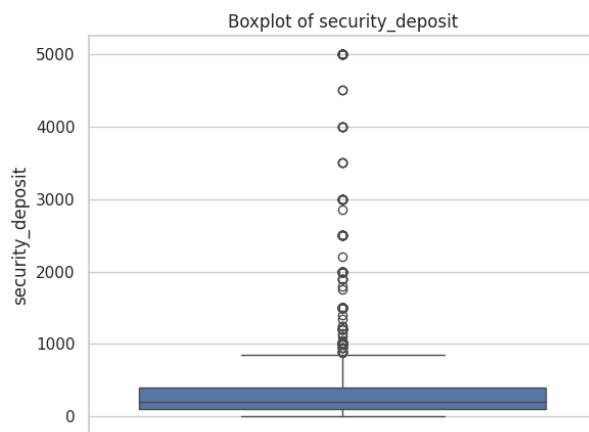
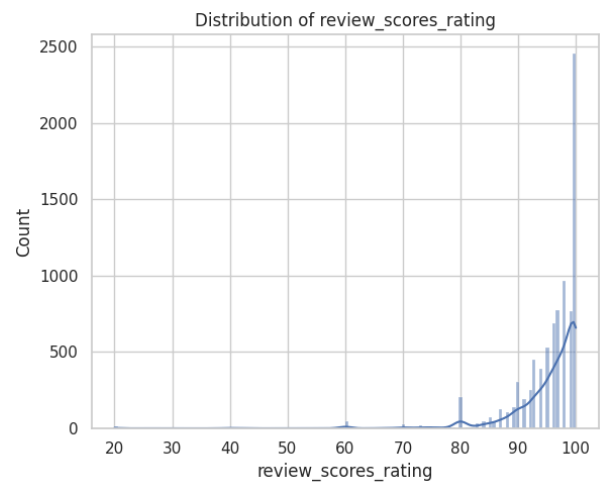
- **Step:** Filled missing values based on feature type:
 - **Numerical Columns:** Missing values in numerical columns such as "beds", "bathrooms", "bedrooms", "cleaning_fee", "security_deposit", "host_total_listings_count", "host_listings_count", "host_response_rate", and "host_since" are filled with the median value of the respective column.
 - **Categorical Columns:** Missing values in categorical columns like "host_response_time", "neighbourhood", "market", "host_location", "host_neighbourhood", "host_identity_verified", "host_has_profile_pic", and "host_is_superhost" are filled with the mode (most frequent value) of the respective column.
 - **Group-Based Imputation:**
 - Missing values in the "zipcode" column are filled using the median value of the "zipcode" for the same "city".

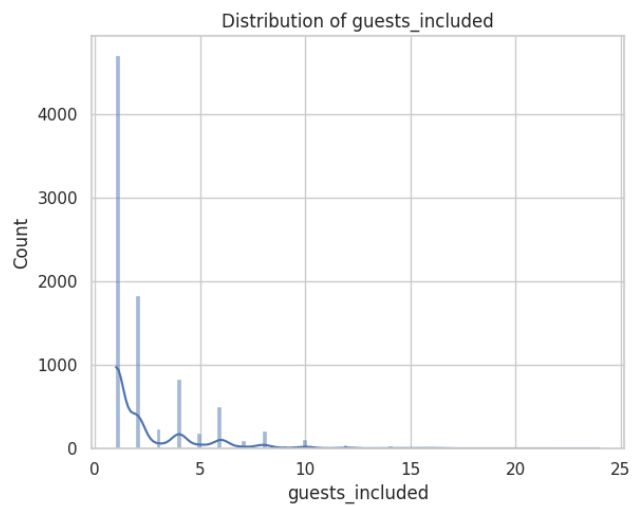
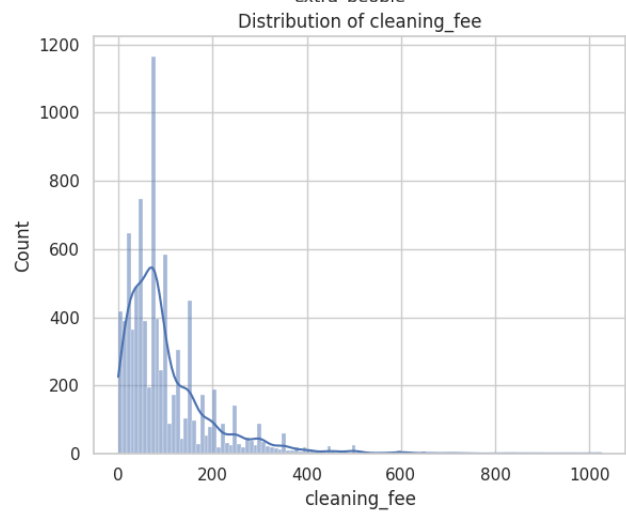
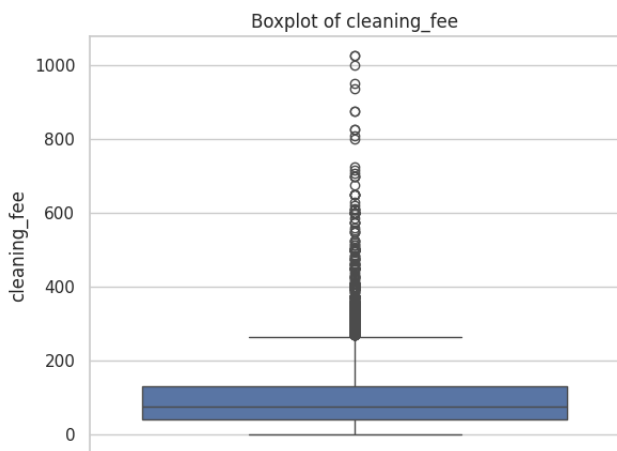
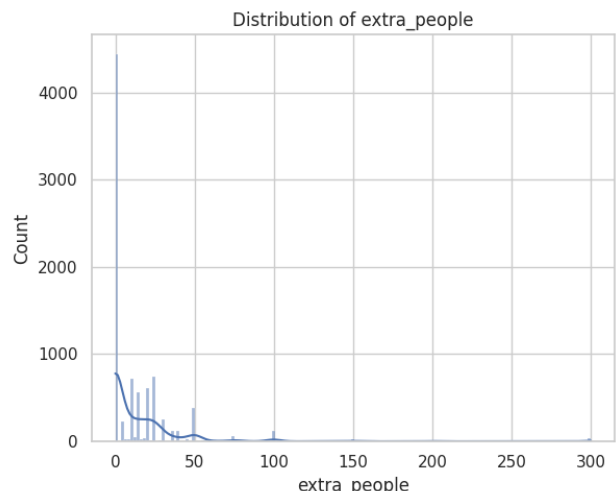
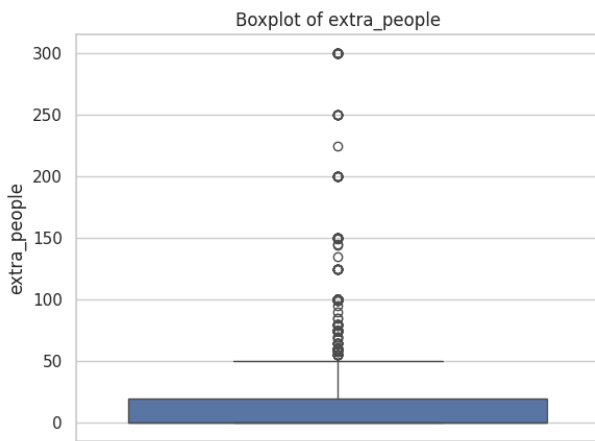
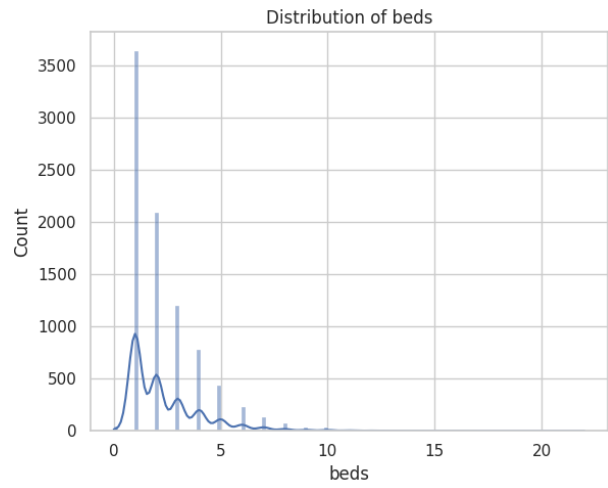
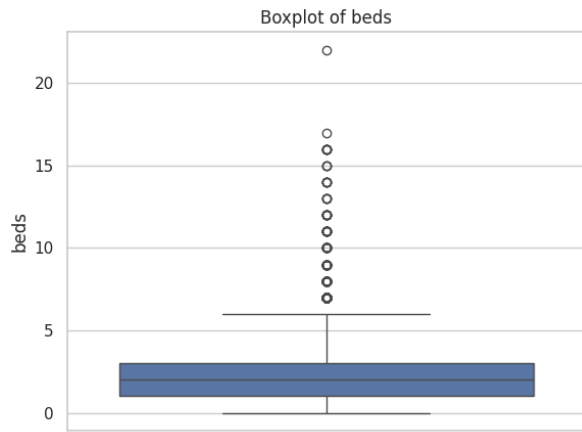
- Missing values in the "state" column are filled by extracting the relevant part from the "street" column.
- **Purpose:** Ensure a complete dataset while preserving relationships between features.

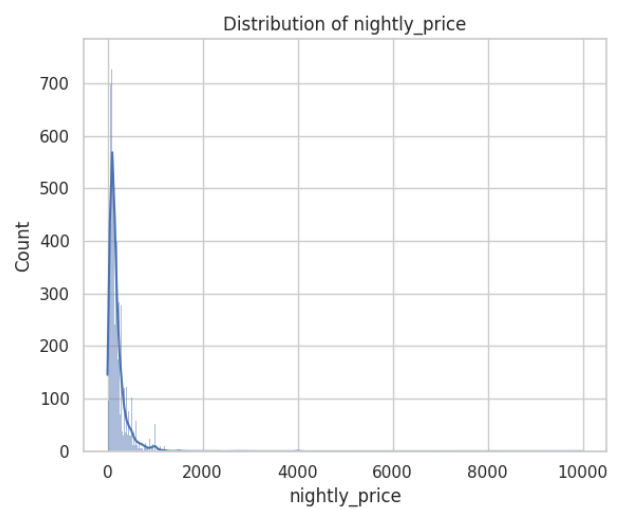
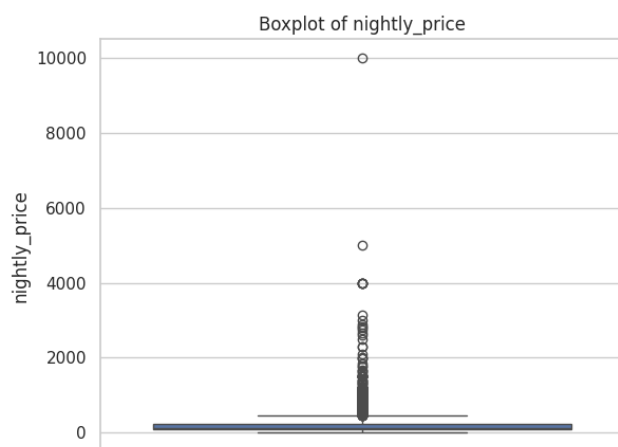
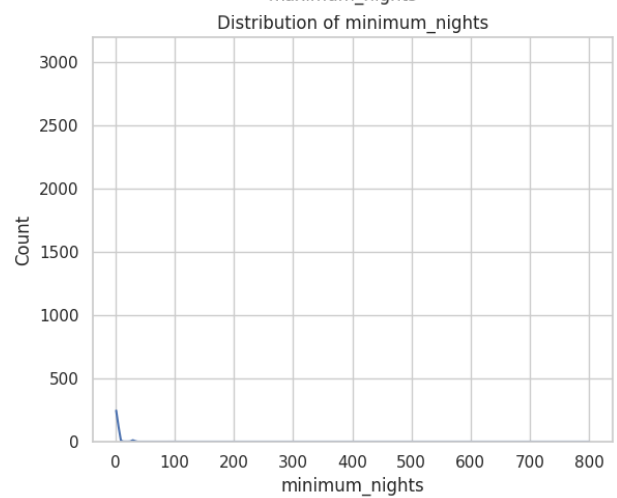
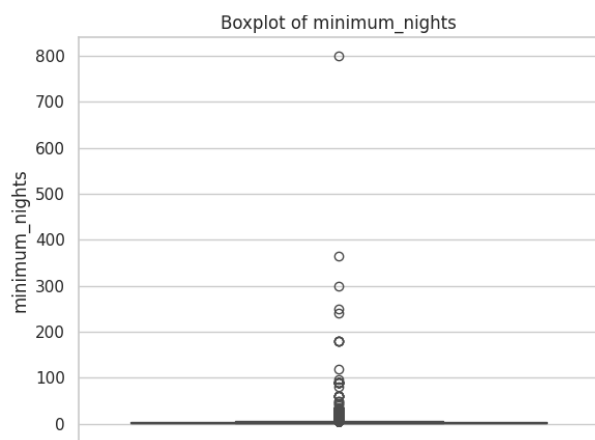
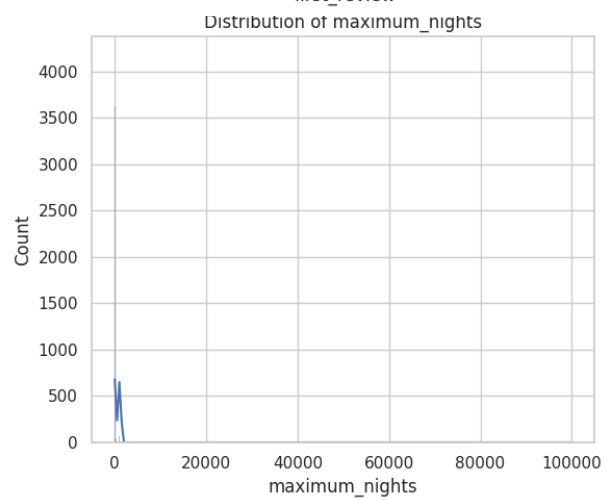
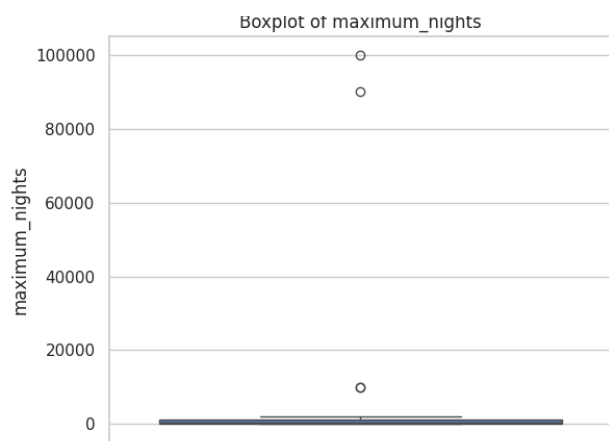
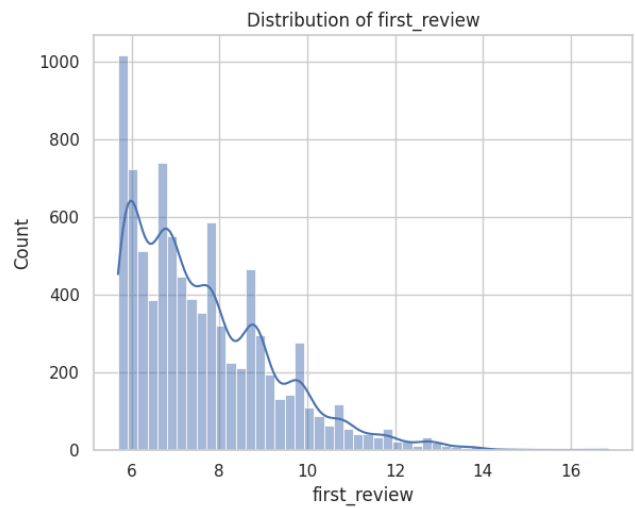
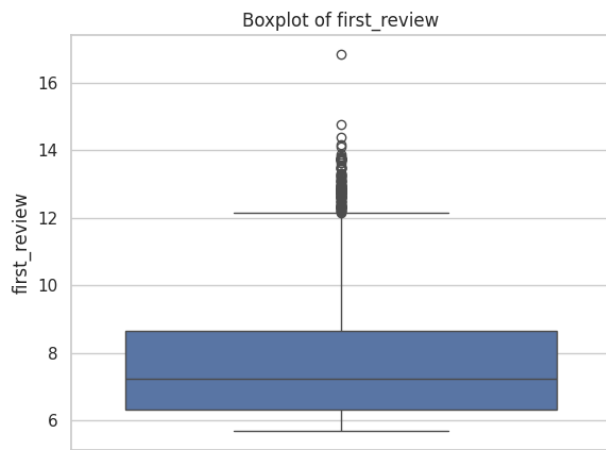
7. Outlier Detection

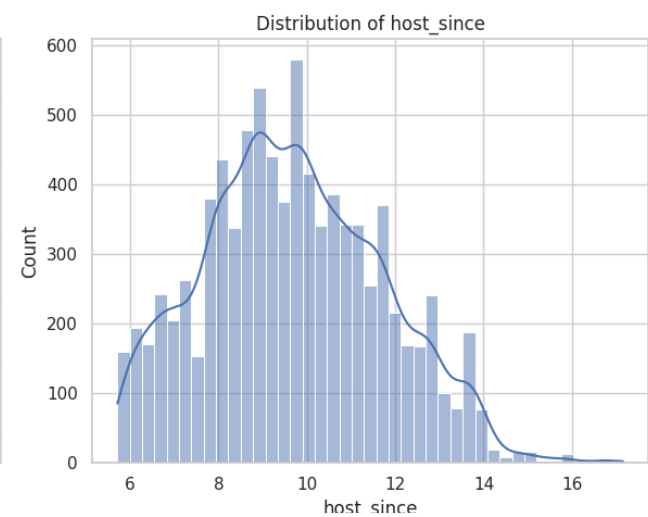
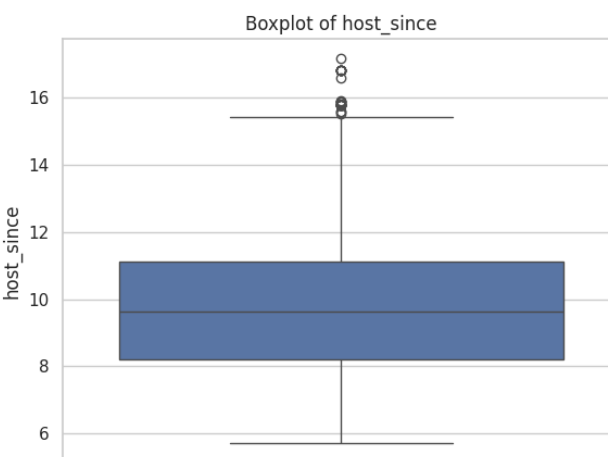
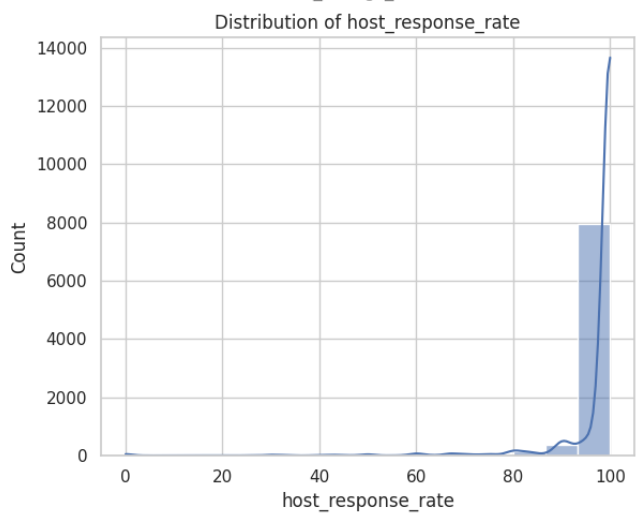
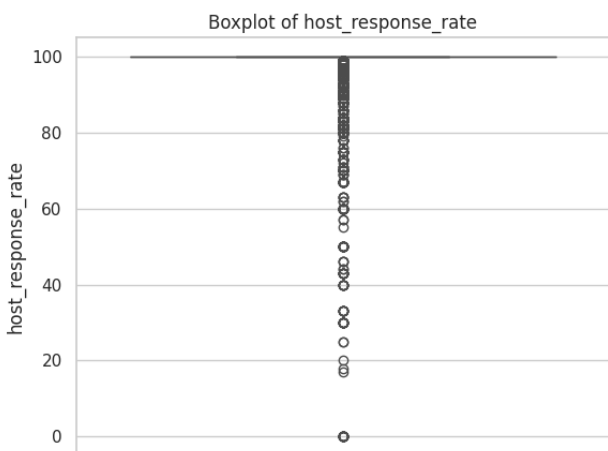
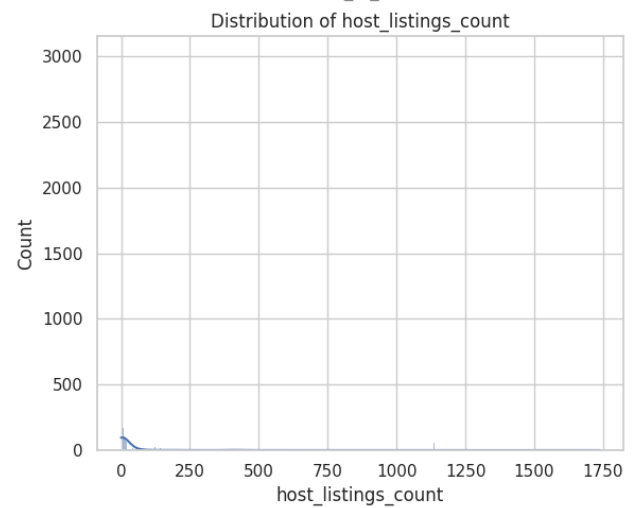
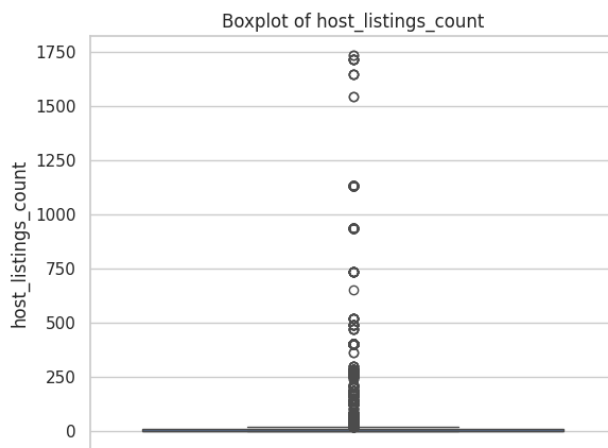
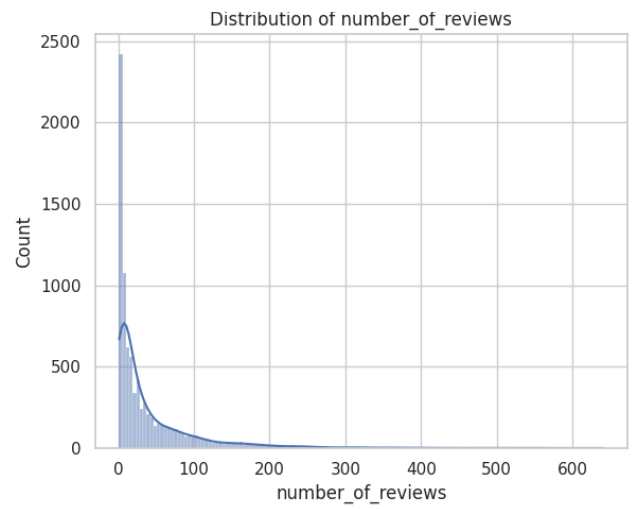
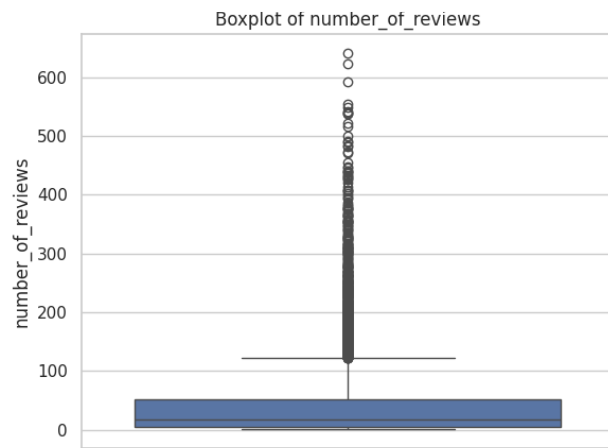
- **Step:** Draw boxplots and distribution plots for numerical features to check outliers, identified using the IQR method.

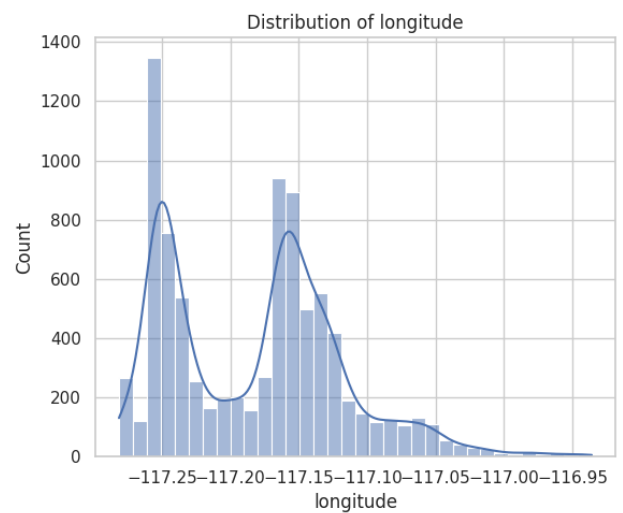
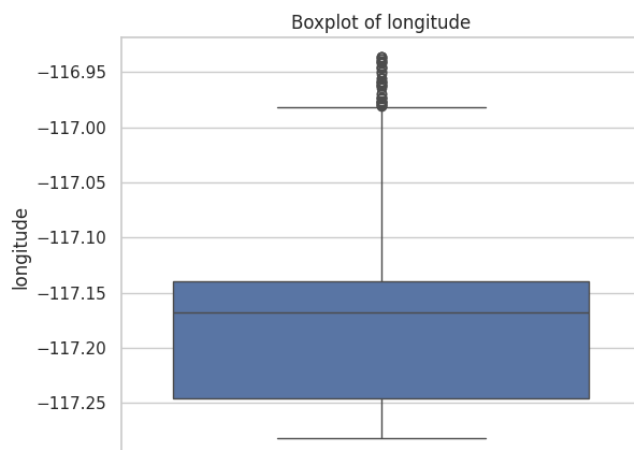
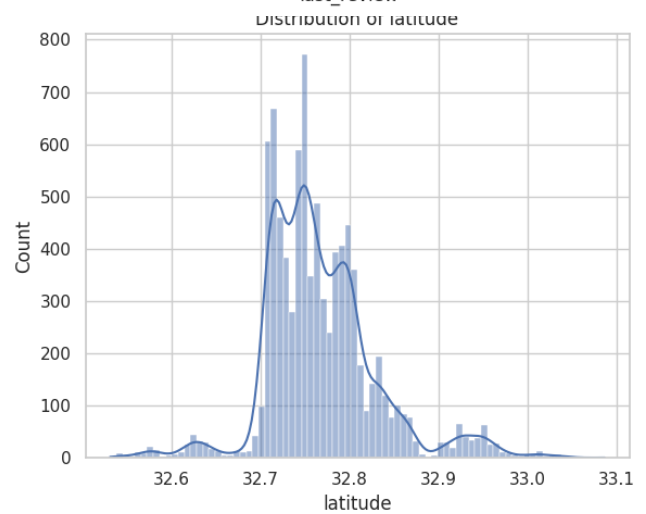
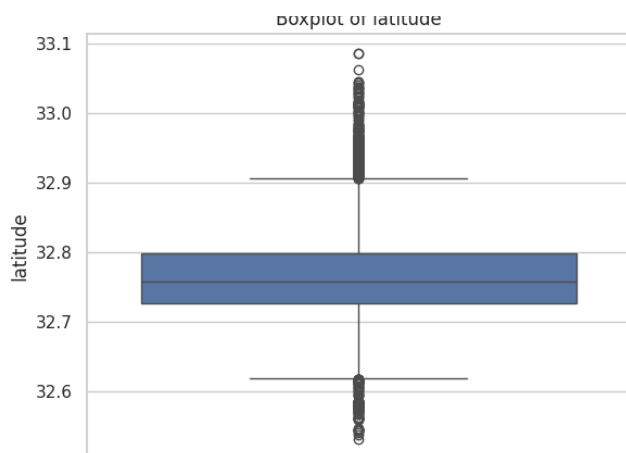
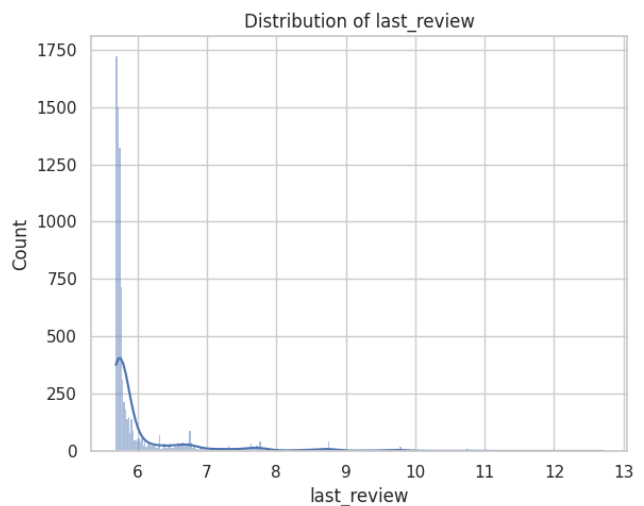
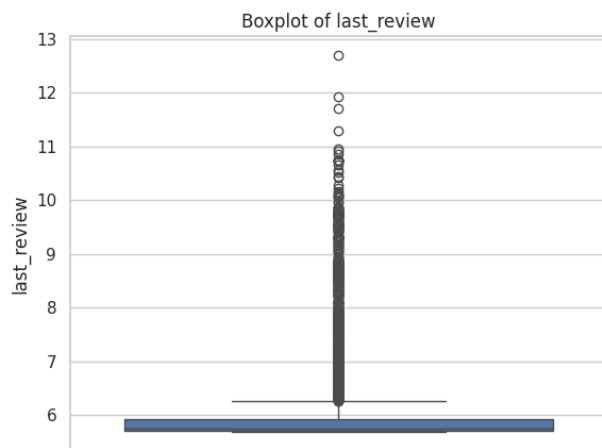
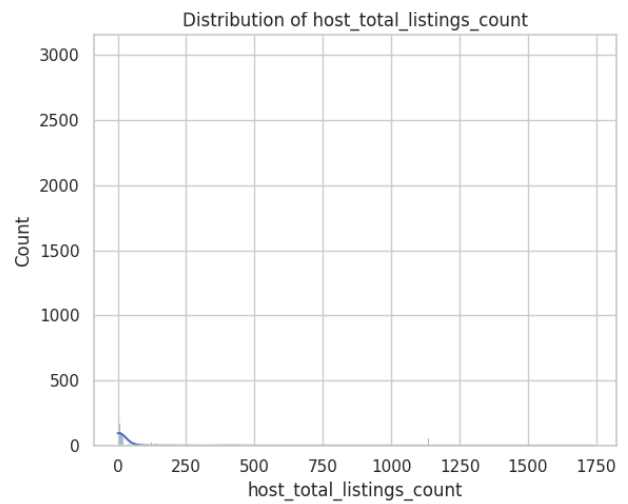
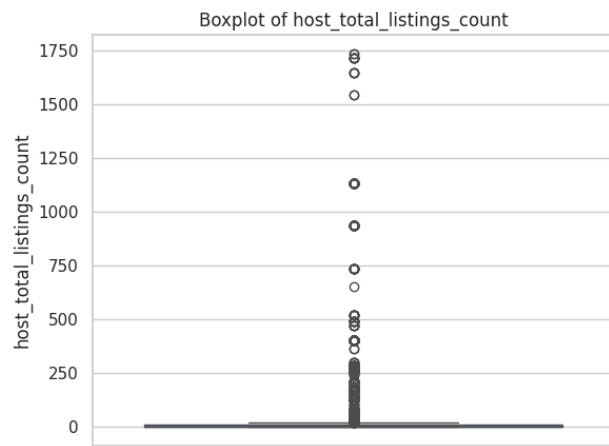












- **Details:** Visualizations helped detect anomalies in the data.
- **Purpose:** Prevent outliers from skewing model performance.

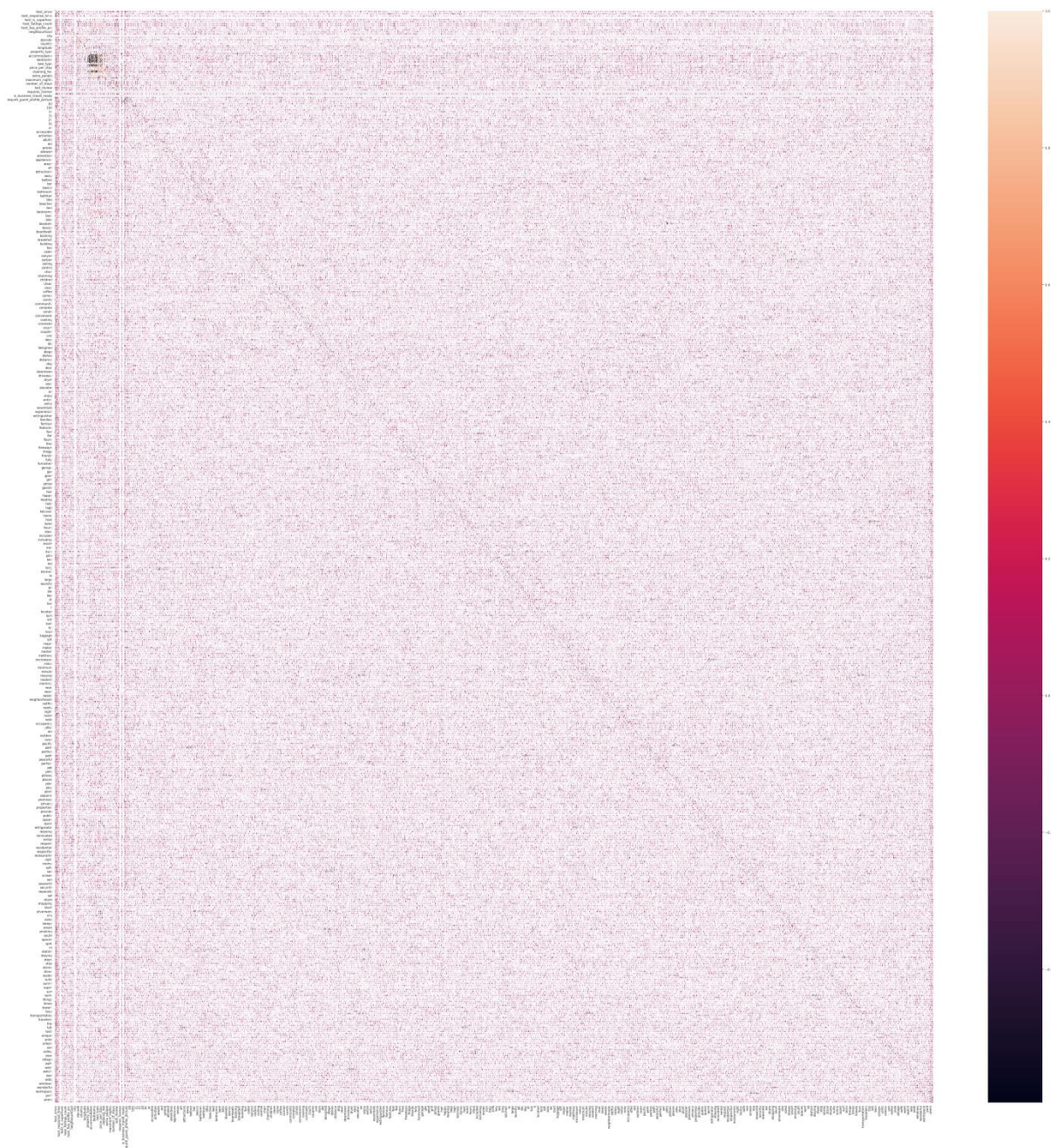
8. Encoding Categorical Variables

- **Step:** Encoded categorical data:
 - **Binary Encoding:** Replaced 't'/'f' with 1/0 in `host_is_superhost`, `host_has_profile_pic`, `host_identity_verified`, `is_location_exact`, `requires_license`, `instant_bookable`, `is_business_travel_ready`, `require_guest_profile_picture`, `require_guest_phone_verification`.
 - **Manual Mapping:** Used dictionaries for `country`, `host_response_time`, `room_type`, `bed_type`, `market`, `cancellation_policy`.
 - **Special Cases:**
 - `state`: Standardized multiple unique values referring to the same place with a single code.
 - `city`: Used a lambda function to map variations in writing style (e.g., case differences) to a standard code.
 - **Factorize Encoding:** Applied to high-cardinality columns: `host_neighbourhood`, `host_location`, `property_type`, `neighbourhood_cleansed`, `neighbourhood`.
 - **TF-IDF Encoding:** Encoded text columns `summary`, `space`, `description`, `neighborhood_overview`, `notes`, `transit`, `access`, `interaction`, `house_rules`, `host_about`, `amenities` with `max_features=500` to extract key words.
- **Purpose:** Transform categorical and text data into numerical formats for modeling.

9. Feature Selection

❖ Correlation Matrix:

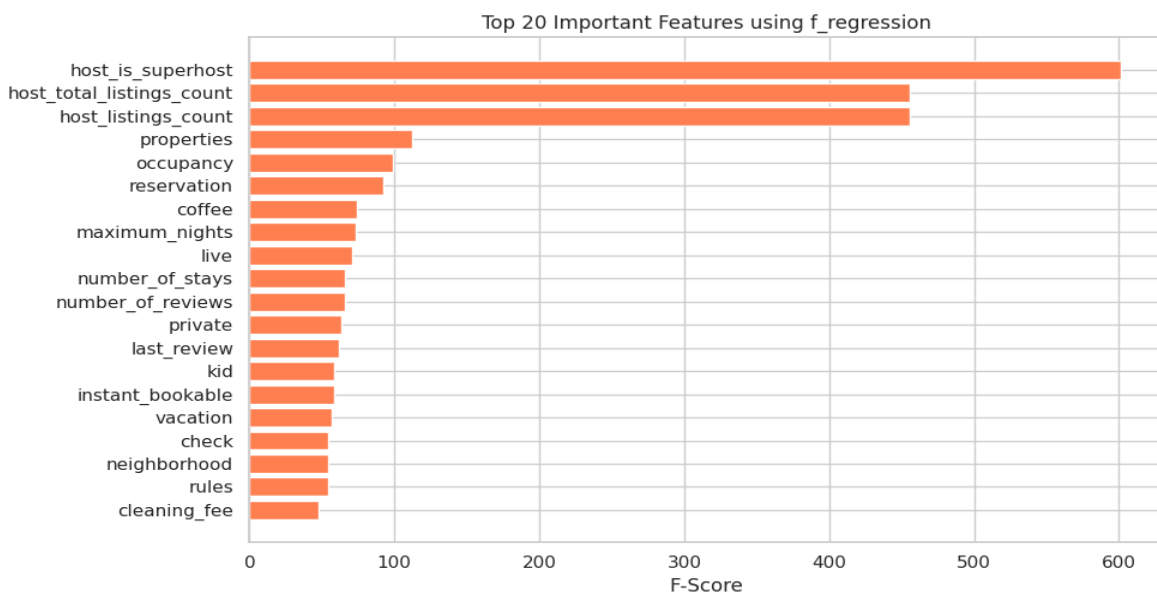
- **Step:** Performed correlation analysis, visualized with a heatmap, and selected top features.
- **Details:** Identified highly correlated features for potential removal. We experimented with different values of `K_features` (80, 60, 10, 30, 130, 150, 100, 120) for feature selection, and the best value was 120.
- **Purpose:** Reduce **multicollinearity** and enhance model performance.



	MSE	R2
Linear Regression	16.370422	0.200837
Lasso Regression	16.388072	0.199976
ElasticNet	16.385637	0.200094
Decision Tree	17.143965	0.163075
XGBoost	16.381313	0.200305
LightGBM	16.315404	0.203523
Random Forest	16.408394	0.198983
Gradient Boosting	16.420732	0.198381

❖ SelectKBest:

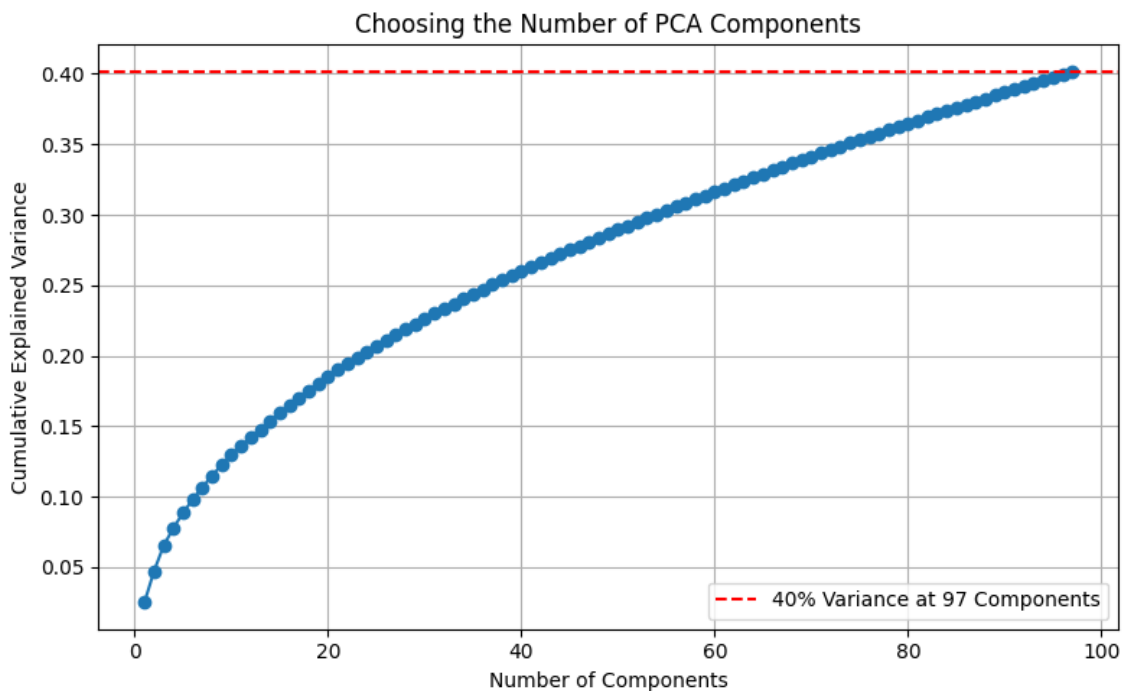
- **Step:** Applied univariate statistical tests to rank features by importance.
- **Details:** Selected the top k features based on their scores using scoring functions like `f_regression`. We experimented with different values of `K_features` (80, 60, 10, 30, 130, 150, 100, 120) for feature selection, and the best value was 120.
- **Purpose:** Retain the most relevant features and reduce dimensionality for improved performance.



	MSE	R2
Linear Regression	16.367643	0.200973
Lasso Regression	16.388081	0.199975
ElasticNet	16.384207	0.200164
Decision Tree	17.143965	0.163075
XGBoost	16.329069	0.202856
LightGBM	16.248348	0.206797
Random Forest	16.817113	0.179031
Gradient Boosting	16.426099	0.198119

❖ PCA (Principal Component Analysis):

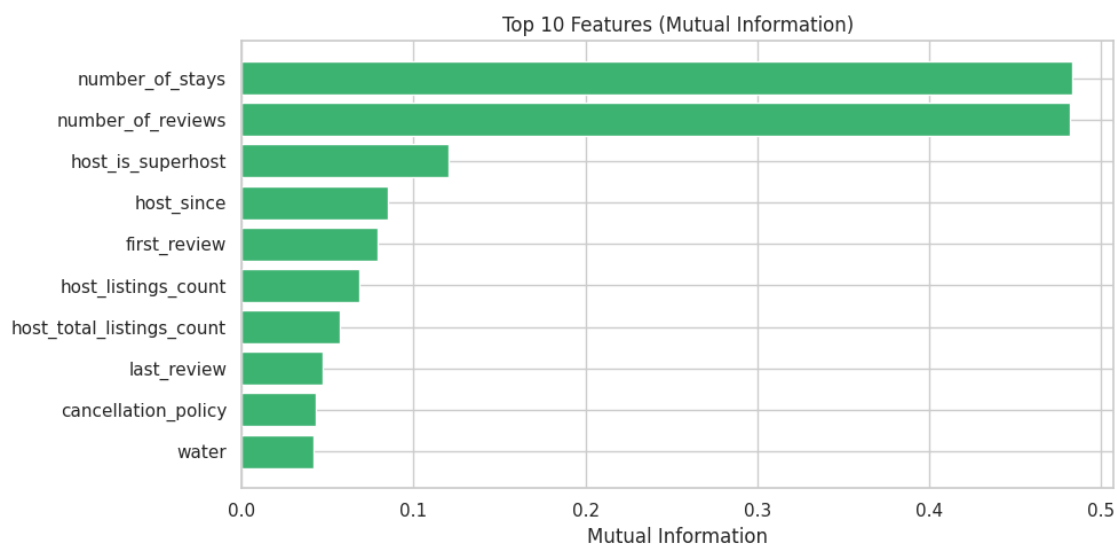
- **Step:** Transformed original features into a set of uncorrelated components ranked by explained variance.
- **Details:** Reduced the dataset dimensionality by selecting components that retained the most variance. We experimented with different values of n_component ratio(80% with 337 component , 60 % with 217 component , 40 % with 97 component)
- **Purpose:** Simplify the model, improve performance, and reduce overfitting.



	MSE	R2
Linear Regression	17.249119	0.157941
Lasso Regression	17.231063	0.158823
ElasticNet	17.231096	0.158821
Decision Tree	19.514369	0.047358
XGBoost	17.258767	0.15747
LightGBM	17.290886	0.155902
Random Forest	17.964762	0.123006
Gradient Boosting	17.331509	0.153919

❖ **Mutual Information Regression:**

- **Step:** Calculated mutual information between each feature and the target variable.
- **Details:** Identified features with strong statistical dependency on the output. We experimented with different values of K (80, 60, 10, 30, 130, 150, 100, 120) for feature selection, and the best value was 120.
- **Purpose:** Select features with the most information about the target for better predictive power.



	MSE	R2
Linear Regression	16.505367	0.194249
Lasso Regression	16.483602	0.195312
ElasticNet	16.491502	0.194926
Decision Tree	17.189954	0.16083
XGBoost	15.544505	0.241156
LightGBM	15.489898	0.243822
Random Forest	15.667363	0.235159
Gradient Boosting	15.931661	0.222256

10. Models

❖ Linear Regression Model:

- **Model Overview:**

The evaluated model is a Linear Regression model, optimized through GridSearch to determine the optimal hyperparameters for enhanced performance.

- **Best Hyperparameters:**

- fit_intercept: True
- n_jobs: None

- **Performance Metrics:**

- **Mean Squared Error (MSE): 16.4912**

This indicates the average squared difference between predicted and true values. A lower MSE is preferable, but the value here suggests moderate prediction errors.

- **R-squared (R^2): 0.1949 (19.49%)**

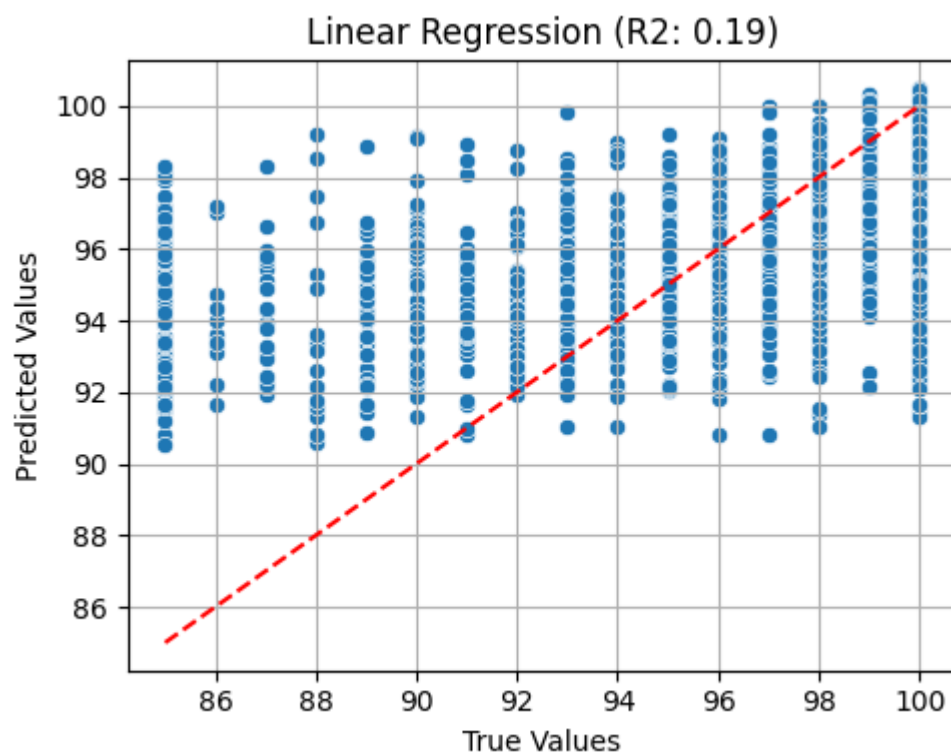
This metric explains the proportion of variance in the dependent variable that is predictable from the independent variables. An R^2 of 0.19 suggests that the model

explains only 19% of the variability in the target data, indicating weak predictive power.

➤ **Mean Absolute Error (MAE): 3.0986**

This represents the average absolute difference between predicted and true values, providing a clearer picture of prediction errors in the original units.

● **Visualization Insights:**



❖ **Lasso Regression Model:**

● **Model Overview:**

The evaluated model is a Lasso Regression, optimized using GridSearch to determine the best hyperparameter for regularization.

● Best Hyperparameter

alpha (Regularization Strength): 0.01

A small alpha value indicates mild regularization, suggesting that the model retains most features but with slight coefficient shrinkage to avoid overfitting.

● Performance Metrics:

➤ Mean Squared Error (MSE): 16.7463:

Slightly higher than the Linear Regression model (16.4912), indicating marginally worse prediction accuracy.

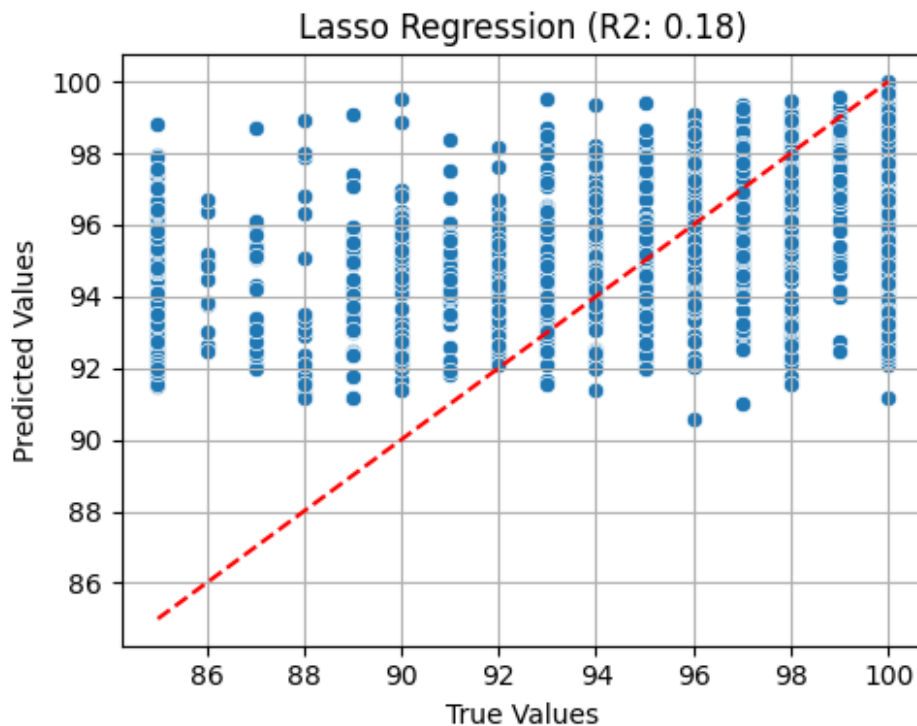
➤ R-squared (R^2): 0.1825 (18.25%):

The model explains 18.25% of the variance in the target variable, showing weak predictive power (even slightly lower than Linear Regression's 19.49%).

➤ Mean Absolute Error (MAE): 3.1178

Comparable to Linear Regression (3.0986), confirming consistent but limited precision.

● Visualization Insights



❖ **Decision Tree Regressor:**

● **Model Overview:**

The evaluated model is a Decision Tree Regressor, optimized using GridSearchCV to identify the best hyperparameters for predictive performance.

Best Hyperparameters:

➤ **max_depth: 3**

Limits tree depth to 3 levels, preventing overfitting but potentially underfitting complex patterns.

➤ **min_samples_split: 2**

Allows splitting nodes with just 2 samples, which may lead to capturing noise in deeper levels.

● **Performance Metrics**

➤ **Mean Squared Error (MSE): 17.1976**

Higher than both Linear Regression (16.49) and Lasso (16.75), indicating worse prediction accuracy.

➤ **R-squared (R^2): 0.1605 (16.05%)**

Explains only 16% of the target variance—weakest performance among tested models so far.

➤ **Mean Absolute Error (MAE): 3.1852**

Slightly higher errors than Linear/Lasso, reinforcing the model's limitations.

● **Visualization Insights:**



❖ XGBoost Regressor:

● Model Overview

The evaluated model is an XGBoost Regressor, an advanced gradient-boosting algorithm optimized via GridSearchCV for hyperparameter tuning. This ensemble method combines multiple weak learners (decision trees) to improve prediction accuracy.

● Best Hyperparameters:

- learning_rate: 0.1 (Controls step size in boosting; balances speed and accuracy)
- max_depth: 5 (Limits tree depth to prevent overfitting while retaining complexity)
- n_estimators: 50 (Number of boosting rounds/trees; higher values improve learning but increase computational cost)

● Performance Metrics

➤ MSE:15.7978

Lowest among the tested models, indicating better accuracy than Linear/Lasso/Decision Tree.

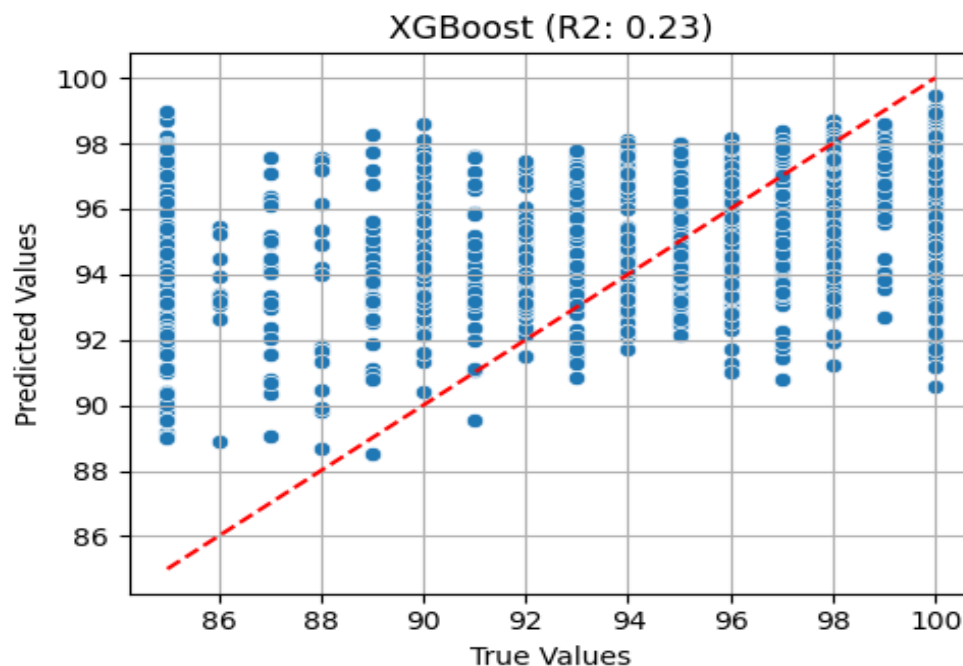
➤ **R²:0.2288**

Explains 22.88% of variance—best so far, but still moderate.

➤ **MAE: 2.9978**

Lowest error (compared to 3.0+ in other models), suggesting improved precision.

● **Visualization Insights:**



❖ **LightGBM Regressor:**

● **Model Overview**

The evaluated model is a LightGBM Regressor, a high-performance gradient-boosting framework optimized for efficiency and accuracy. The model was tuned using GridSearchCV to identify optimal hyperparameters. Despite warnings during training about splits with no positive gain, the model achieved competitive results.

● **Best Hyperparameters**

➤ **learning_rate: 0.1** (Balances training speed and model performance)

➤ **max_depth: 3** (Restricts tree depth to prevent overfitting)

- **n_estimators: 100** (Number of boosting iterations; higher values improve learning at the cost of computational resources)

Metric: **Value Interpretation**

MSE: 15.6931

Lowest among all tested models, indicating superior prediction accuracy.

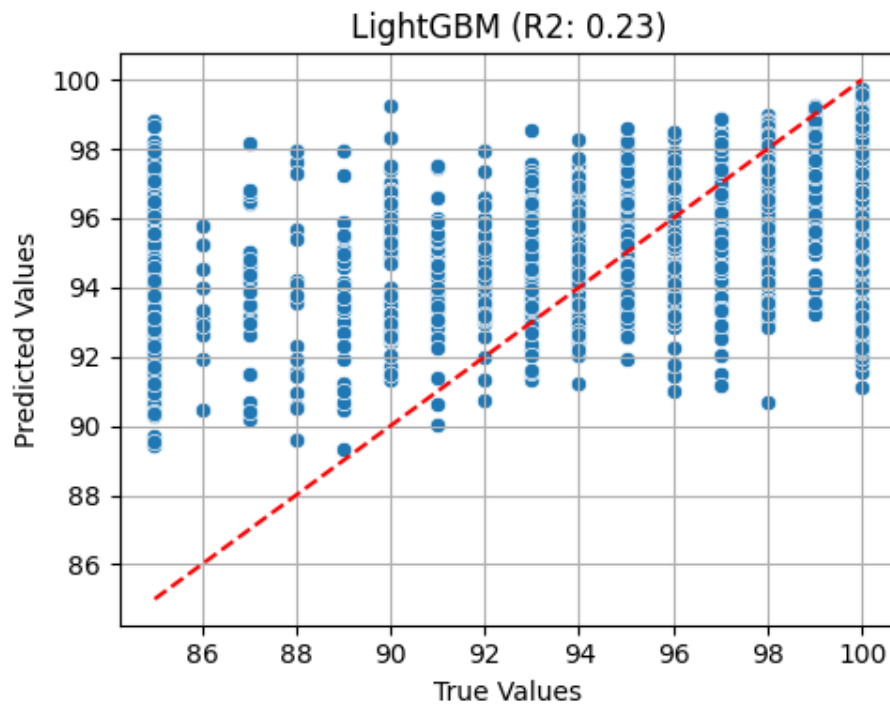
R²: 0.2339

Explains 23.39% of variance—best performance so far, though still moderate.

MAE: 2.9969

Lowest error across all models, demonstrating improved precision.

- **Visualization Insights:**



❖ Random Forest Regressor:

● Model Overview

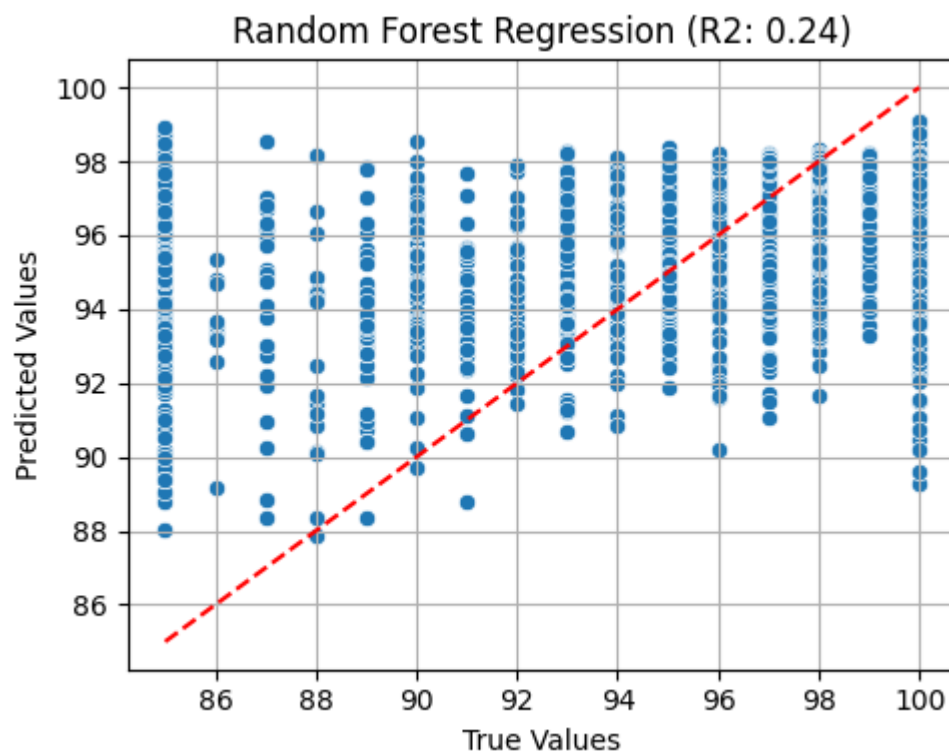
The evaluated model is a Random Forest Regressor, an ensemble method that combines multiple decision trees to improve predictive accuracy and reduce overfitting. The model was optimized using GridSearchCV to determine the best hyperparameters.

● Best Hyperparameters

- max_depth: 10 (Allows trees to grow deeper, capturing more complex patterns while avoiding excessive overfitting)
- n_estimators: 200 (Number of trees in the forest; higher values generally improve performance but increase computational cost)

Performance Metrics

- MSE: 15.6402 Lowest among all tested models, indicating superior prediction accuracy.
- R^2 : 0.2365, Explains 23.65% of variance—best performance so far, though still moderate.
- MAE: 2.9701, Lowest error across all models, demonstrating the highest precision.
- Visualization Insights:



❖ Gradient Boosting Regressor:

● Model Overview

The evaluated model is a Gradient Boosting Regressor, an advanced ensemble technique that builds trees sequentially to correct errors from previous iterations. The model was optimized using GridSearchCV to determine the most effective hyperparameters.

- Best Hyperparameters

- learning_rate: 0.1

(Controls contribution of each tree; balances speed and accuracy)

- max_depth: 3

(Limits tree depth to prevent overfitting while maintaining model complexity)

- n_estimators: 100

(Number of boosting stages; higher values improve learning at computational cost)

- Performance Metrics

- MSE:16.007,

Higher than Random Forest (15.64) but competitive with other models.

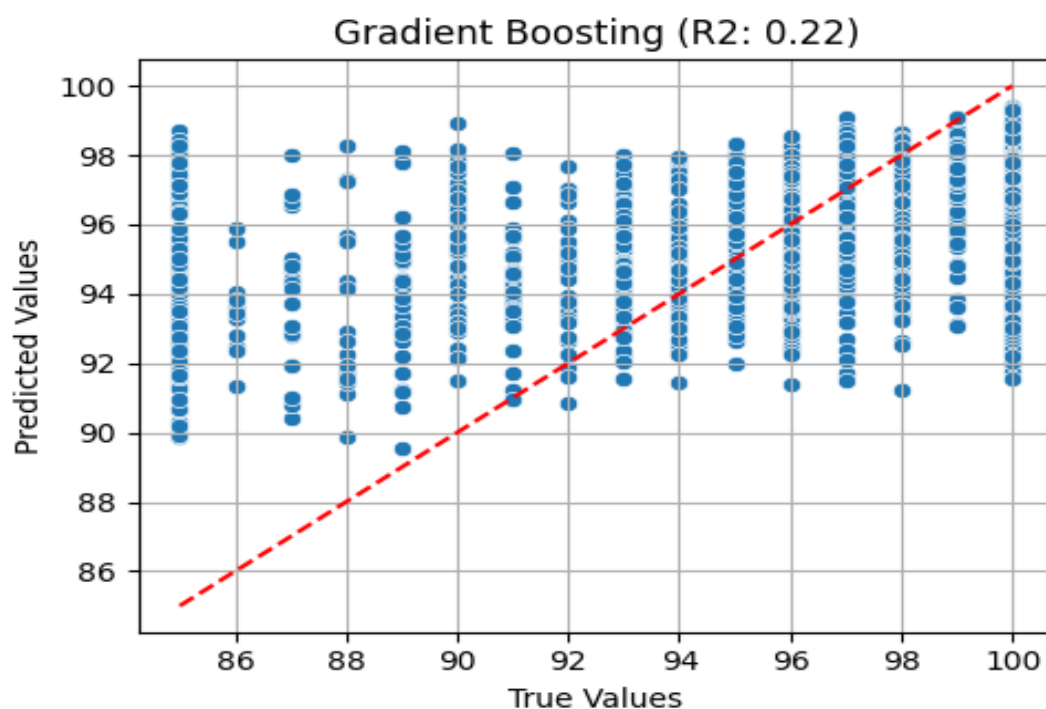
- R²: 0.2186,

Explains 21.86% of variance—performs worse than Random Forest (0.2365).

- MAE:3.0148,

Slightly higher error than top-performing models (Random Forest: 2.9701).

- Visualization Insights:



❖ CatBoost Regressor:

● Model Overview

The evaluated model is a CatBoost Regressor, a high-performance gradient boosting algorithm specifically designed to handle categorical features efficiently. The model was optimized using GridSearchCV to identify the best hyperparameters for optimal performance.

● Best Hyperparameters

➤ depth: 8

(Tree depth, allowing the model to capture complex patterns)

➤ iterations: 200

(Number of boosting rounds, providing sufficient learning capacity)

➤ learning_rate: 0.05

(Smaller step size for more precise optimization)

● Performance Metrics

➤ MSE: 15.4697,

Lowest among all tested models, indicating superior prediction accuracy.

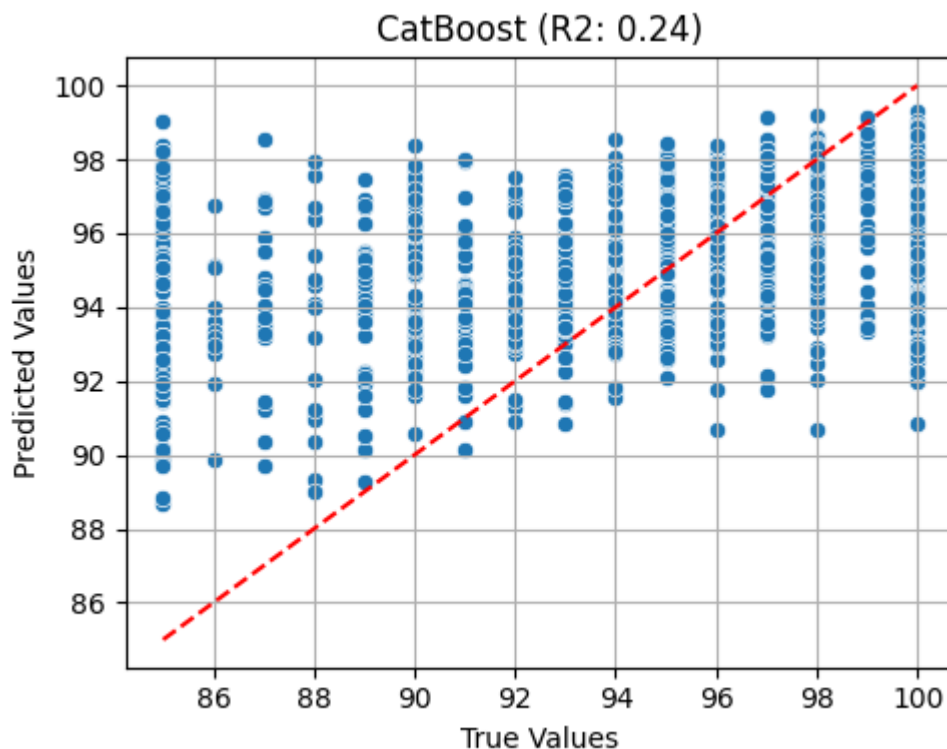
➤ R^2 :0.2448,

Explains 24.48% of variance—best performance achieved so far.

➤ MAE: 2.9551,

Lowest error across all models, demonstrating exceptional precision.

- Visualization Insights:



- ❖ ElasticNet Regression

Model Overview

The evaluated model is an ElasticNet Regression, a linear model that combines L1 (Lasso) and L2 (Ridge) regularization to balance feature selection and coefficient shrinkage. The model was optimized using GridSearchCV to determine the best hyperparameters.

- Best Hyperparameters

- alpha: 0.01 (Overall regularization strength - weak penalty)
- l1_ratio: 0.2 (Balance between L1/L2 regularization: 20% Lasso, 80% Ridge)

- Performance Metrics

- MSE:16.8086,

Higher than tree-based models, indicating weaker predictive accuracy

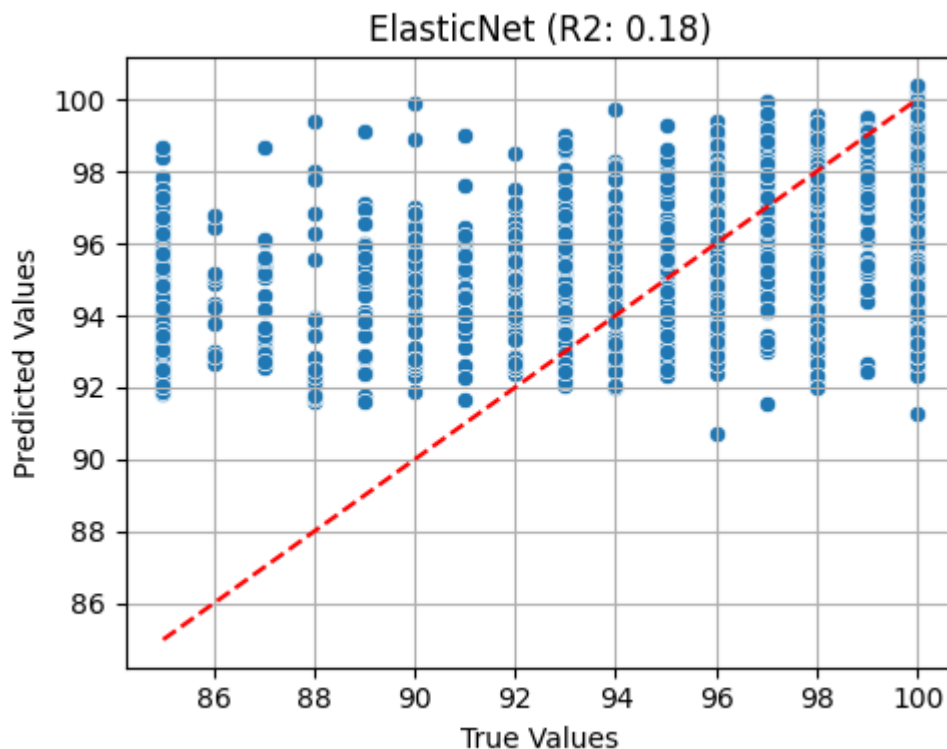
- R²:0.1794,

Explains only 17.94% of variance - poorest performance among tested models

➤ MAE:3.1023,

Error magnitude similar to other linear models (Lasso/Linear Regression)

● Visualization Insights:



11. Conclusion

The pipeline prepares the dataset comprehensively for predicting guest satisfaction. It involves loading, cleaning, imputing missing values, encoding categorical variables, and selecting features. The result is a cleaned and fully encoded dataset, ready for model training. The model evaluated is a Linear Regression model, optimized using GridSearch to identify the best hyperparameters for performance.

Training and evaluation of machine learning models will be the next step in our project.

The best feature selection method was (Mutual Information), with the optimal number of features selected at $k = 120$.

The best-performing model identified was (*CatBoost*) among the 9 models we tried.

Changes in MS1

1.Preprocessing

Drop Columns `host_listings_count` because have same similarity between Columns `host_total_listings_count`

Outliers

Clipping Extreme Values in Outliers

2.Feature Selection

SelectKBest with `f_regression`: Selects the top 50 features with the highest F-scores.

SelectKBest with `mutual_info_regression`: selects the top 50 features based on mutual information scores.

Feature Importance with `XGBoost`: Selects the top **50 features** with the highest importance scores.

SelectFromModel with `LassoCV`.

combines the features selected by all four methods using a **union** operation
(`set(selected_features_f) | set(selected_features_mi) | set(selected_features_xgb) | set(selected_features_lasso)`).

This ensures that **all features deemed important by at least one method** are included in the final feature set (`all_selected_features`).

3.Models

❖ CatBoost Regressor:

● Model Overview

The evaluated model is a CatBoost Regressor, a high-performance gradient boosting algorithm specifically designed to handle categorical features efficiently. The model was optimized using GridSearchCV to identify the best hyperparameters for optimal performance. The model was optimized using **GridSearchCV** to explore combinations of key hyperparameters in a time-efficient manner.

● Best Hyperparameters

- **iterations:** 200

More boosting rounds for better learning.

- **learning_rate:** 0.05

A smaller learning rate ensures gradual convergence and better generalization.

- **depth:** 6

Deeper trees allow modeling of more complex patterns in the data.

- **l2_leaf_reg:** 3

Regularization strength to control overfitting.

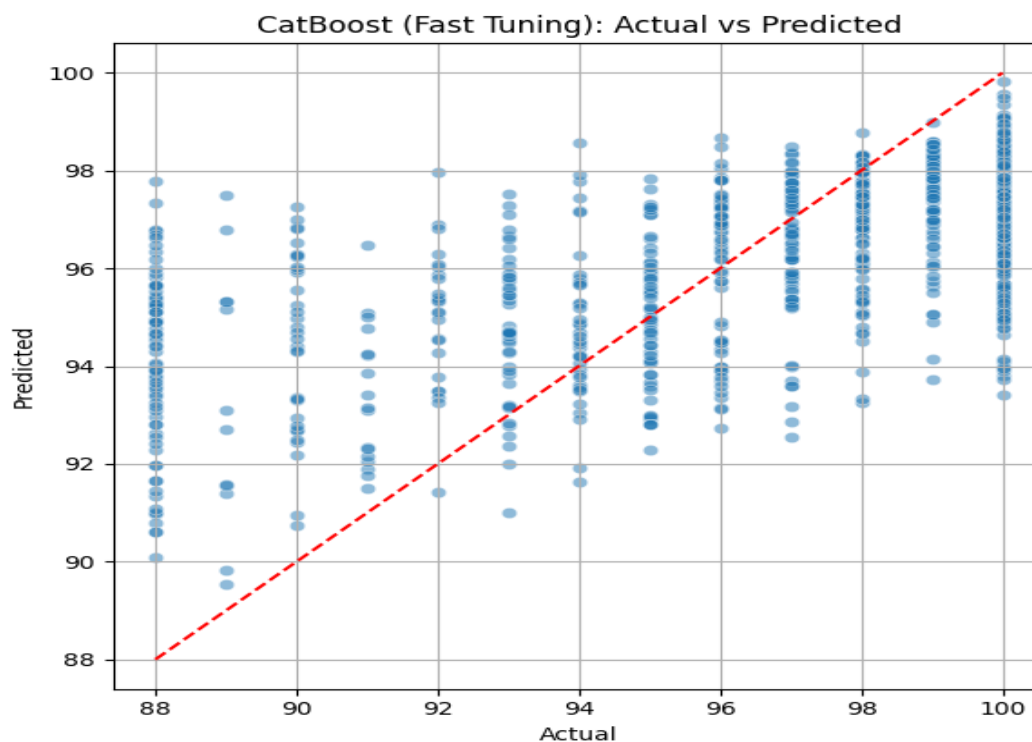
- **subsample:** 0.8

Prevents overfitting by training on random subsets of data in each iteration.

Performance Metrics

- MSE: 9.7245
- R^2 : 0.3655

● Visualization Insights:



❖ Linear Regression Model:

● Model Overview:

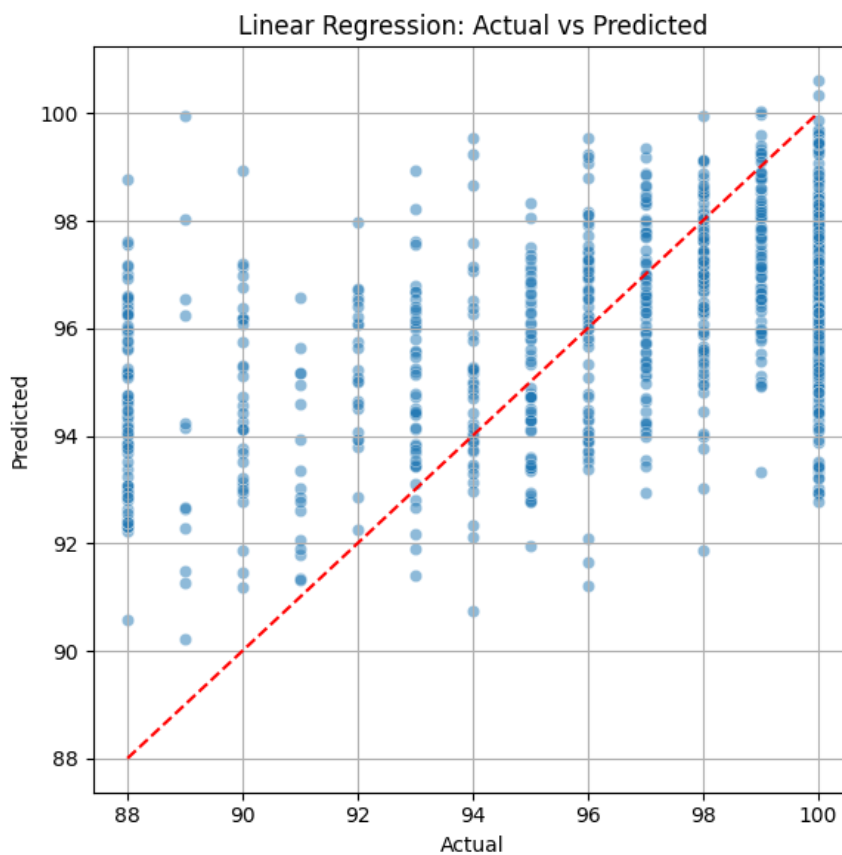
The evaluated model is a Linear Regression model, optimized through GridSearch to determine the optimal hyperparameters for enhanced performance.

● Performance Metrics:

➤ Mean Squared Error (MSE): 12.1321

➤ R-squared (R^2): 0.2085

● Visualization Insights



❖ Lasso Regression Model:

● Model Overview:

The evaluated model is a Lasso Regression, optimized using GridSearch to determine the best hyperparameter for regularization.

● Best Hyperparameter

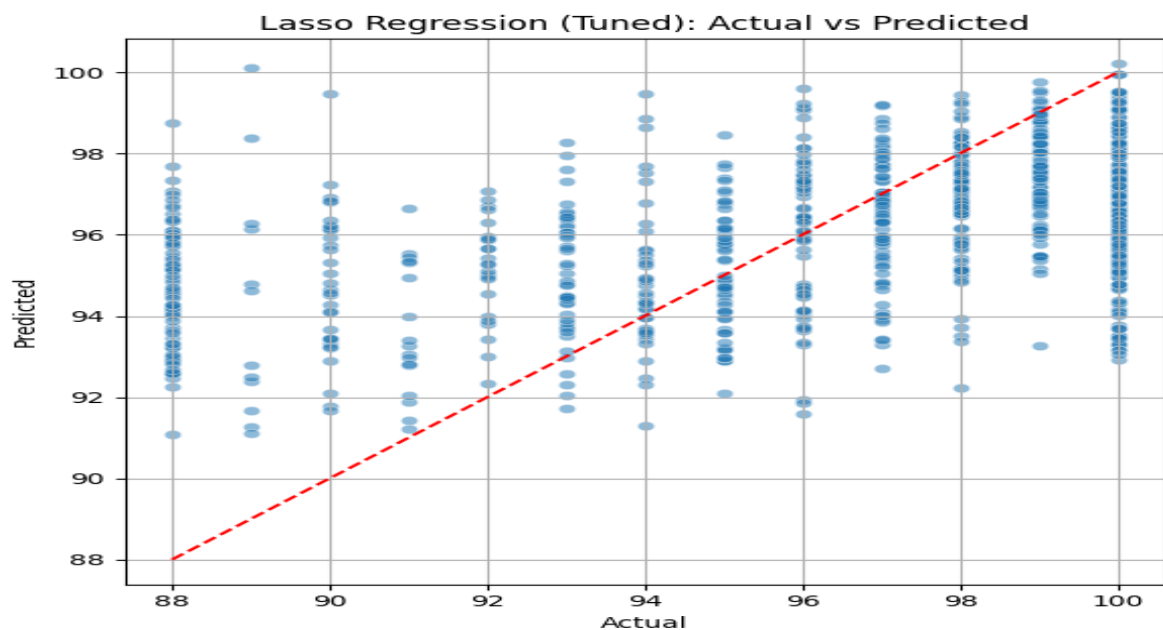
Best Alpha:0.001

● Performance Metrics:

➤ Mean Squared Error (MSE): 12.1723

➤ R-squared (R^2): 0.2058

● Visualization Insights



❖ Decision Tree Regressor:

● Model Overview:

The evaluated model is a Decision Tree Regressor, optimized using GridSearchCV to identify the best hyperparameters for predictive performance.

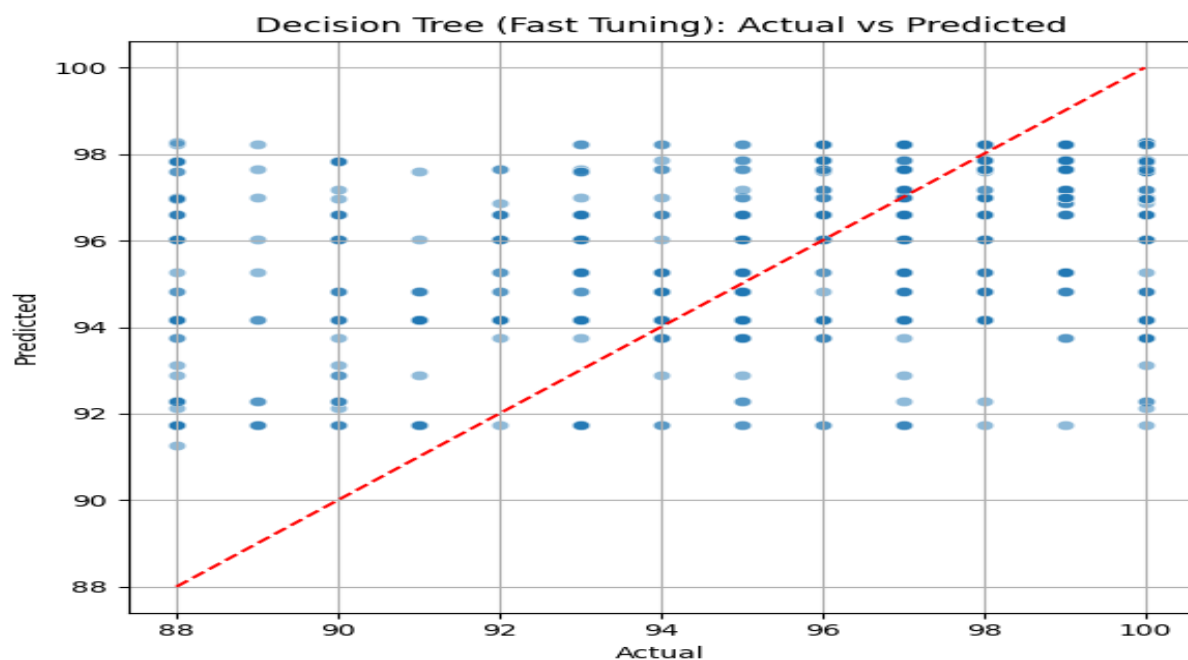
Best Hyperparameters:

- max_depth: 5 ,A value of 5 keeps the model simple and generalizable.
- min_samples_split:
- min_samples_split:2
- min_samples_leaf:2 , The minimum number of samples required to split an internal node. The default value (2) allows maximum flexibility.
- max_features: None ,All features are considered when looking for the best split. This often gives a better fit in small datasets

● Performance Metrics

- MSE:11.957
- R^2 : 0.2199

● Visualization Insights:



❖ XGBoost Regressor:

● Model Overview

The evaluated model is an XGBoost Regressor, an advanced gradient-boosting algorithm optimized via GridSearchCV for hyperparameter tuning. This ensemble method combines multiple weak learners (decision trees) to improve prediction accuracy.

● Best Hyperparameters:

➤ learning_rate:0.05

A smaller learning rate ensures gradual convergence and better generalization.

➤ max_depth: 6

Deeper trees allow modeling of more complex patterns in the data.

➤ n_estimators: 100

With a small learning rate (0.05), 100 trees is enough for stable learning without taking too long or overfitting.

➤ subsample: 0.8

Prevents overfitting by training on random subsets of data in each iteration.

➤ Colsample_bytree:0.8

➤ gamma:0

➤ reg_alpha:0

➤ Reg_lambda:1

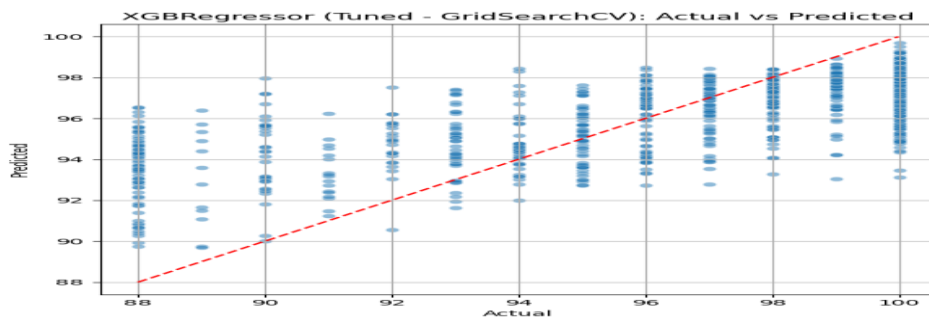
Helps reduce model complexity and prevent overfitting.

● Performance Metrics

➤ MSE:8.4801

➤ R²:0.4467

● Visualization Insights:



❖ LightGBM Regressor:

● Model Overview

The evaluated model is a LightGBM Regressor, a high-performance gradient-boosting framework optimized for efficiency and accuracy. The model was tuned using GridSearchCV to identify optimal hyperparameters. Despite warnings during training about splits with no positive gain, the model achieved competitive results.

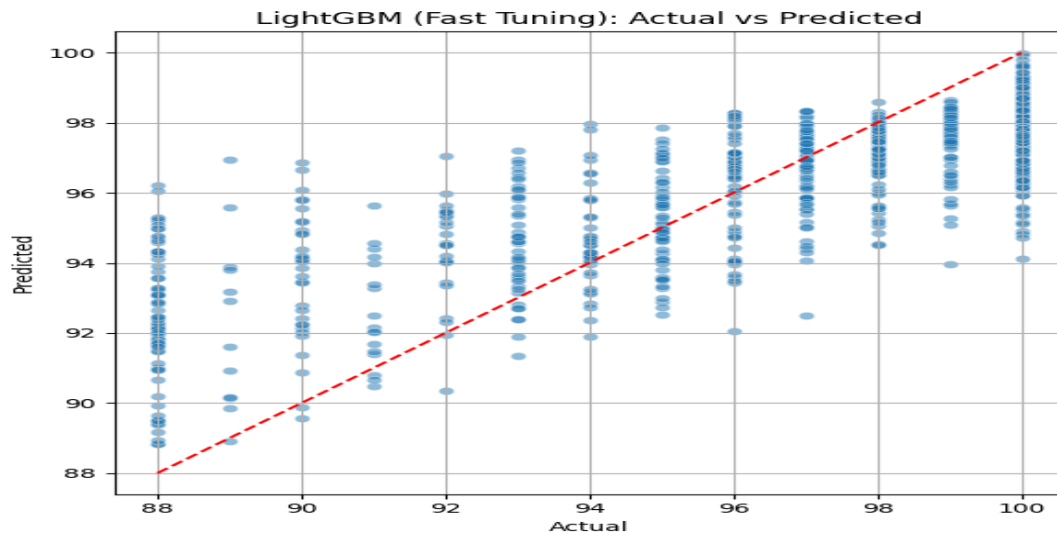
● Best Hyperparameters

- learning_rate:0.05 A smaller learning rate ensures gradual convergence and better generalization.
- max_depth: 10 Depth 10 offers a balance between expressiveness and avoiding overfitting.
- n_estimators: 100
- num_leaves:50 capture detail without overfitting.
- min_child_samples:10 This parameter prevents overly specific splits by requiring a minimum number of data points in leaf nodes. It helps reduce overfitting and ensures stability.
- subsample:0.8 introduces variability that improves generalization and combats overfitting.
- colsample_bytree:0.8 introduces helpful randomness and reduces the likelihood of overfitting.

● Performance Metrics

- MSE:6.0398
- R²:0.6059

● Visualization Insights:



❖ ElasticNet Regression

Model Overview

The evaluated model is an ElasticNet Regression, a linear model that combines L1 (Lasso) and L2 (Ridge) regularization to balance feature selection and coefficient shrinkage. The model was optimized using GridSearchCV to determine the best hyperparameters.

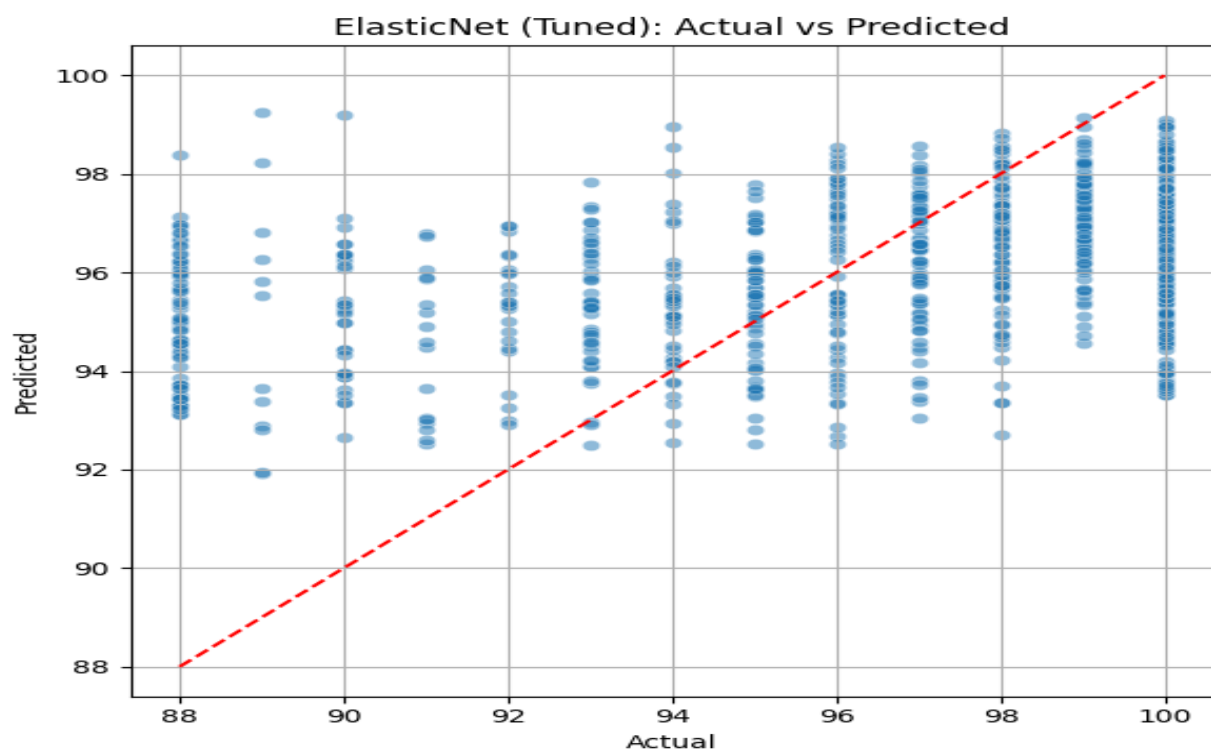
- Best Hyperparameters

- alpha: 0.1
- l1_ratio: 0.5

- Performance Metrics

- MSE:12.7896
- R²:0.1656

- **Visualization Insights:**



❖ Random Forest Regressor:

● Model Overview

The evaluated model is a Random Forest Regressor, an ensemble method that combines multiple decision trees to improve predictive accuracy and reduce overfitting. The model was optimized using GridSearchCV to determine the best hyperparameters.

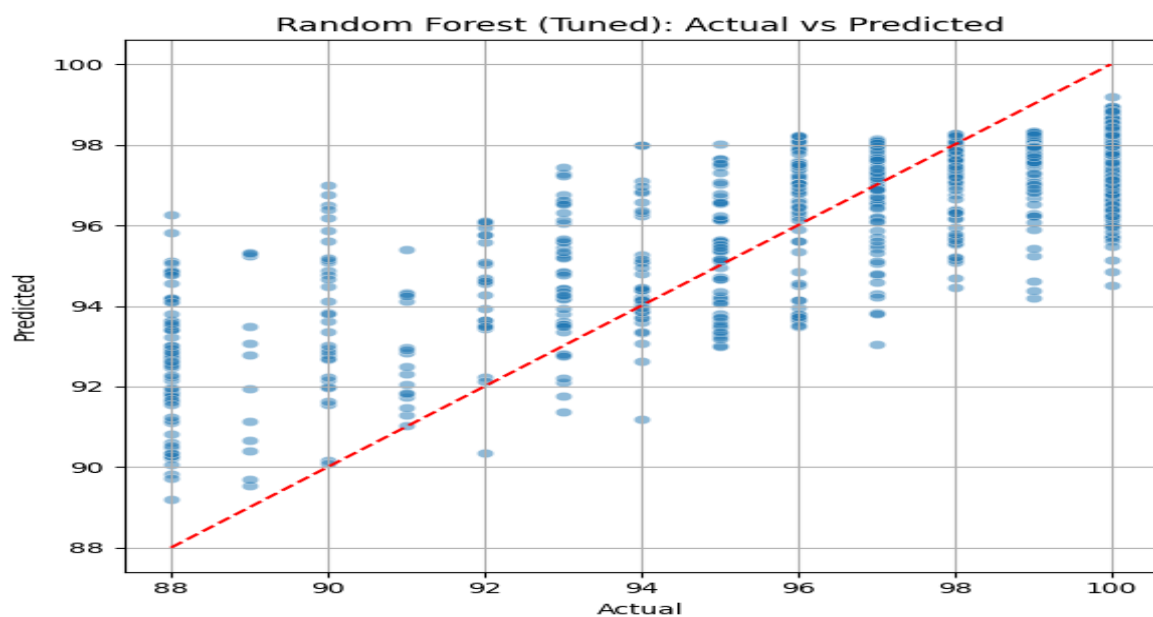
● Best Hyperparameters

- max_depth: 10 (Allows trees to grow deeper, capturing more complex patterns while avoiding excessive overfitting)
- n_estimators: 200 (Number of trees in the forest; higher values generally improve performance but increase computational cost)

Performance Metrics

- MSE: 6.3257 Lowest among all tested models, indicating superior prediction accuracy.
- R^2 : 0.5873 Explains 23.65% of variance—best performance so far, though still moderate.

● Visualization Insights:



(Milestone 2)

1. Introduction

This report extends the pipeline from Milestone 1, focusing on **train-test splitting**, **preprocessing consistency**, and **artifact serialization** for deployment. The dataset GuestSatisfactionPredictionMilestone2.csv is processed to ensure reproducibility and avoid data leakage while preparing for model training.

2. Data Splitting

Step: The dataset was divided into **training (80%)** and **testing (20%)** subsets using stratified sampling based on the target variable guest_satisfaction.

Purpose: Ensure the test set retains the original class distribution of the target variable and guarantees unbiased model evaluation.

3. Preprocessing Replication

All preprocessing steps from Milestone 1 were **reapplied separately** to the training and test sets:

- **Missing Value Handling:**
 - Columns with >80% missing values (e.g., `thumbnail_url`) were dropped.
 - Numerical and categorical missing values were imputed using **training set statistics** (medians and modes) to prevent data leakage.
- **Data Cleaning:**
 - Irrelevant columns (e.g., `id`, `street`) were removed.
 - Date columns (`host_since`, `first_review`) were converted to numerical features (days since current date).

- Special characters (e.g., "\$", "%") were stripped from numerical/text columns.
 - **Outlier Treatment:**
 - Outlier bounds (derived from the training set using the IQR method) were applied to cap extreme values in both splits.
-

4. Feature Encoding

A. Categorical Variables

- **Binary Features:** Columns like `host_is_superhost` were mapped to `0/1` using training data definitions.
- **Manual Mapping:** Dictionaries for `room_type`, `cancellation_policy`, and other categorical features were created from the training set and applied to both splits.
- **High-Cardinality Columns:** Labels for `property_type` and `neighbourhood` were factorized using training data indices.

B. Text Features

- **TF-IDF Encoding:** Text columns (e.g., `amenities`, `summary`) were vectorized using a `TfidfVectorizer` fitted exclusively on the training set.

C. Target Variable Encoding

- The `guest_satisfaction` labels were converted to numerical indices using a mapping derived from the **training set's unique classes**. This mapping ensured consistency across both splits.
-

5. Preprocessing Artifact Serialization

All preprocessing logic and dependencies were saved to a `preprocessing.pkl` file for deployment. This included:

- **Imputation Parameters:** Medians (numerical) and modes (categorical) from the training set.
 - **Outlier Bounds:** IQR-derived thresholds for capping numerical features.
 - **Encoding Mappings:** Binary, manual, and factorization dictionaries, as well as TF-IDF vectorizer weights.
 - **Date Conversion Logic:** Functions to standardize date columns.
 - **Target Variable Mapping:** Label-to-index correspondence for `guest_satisfaction`.
-

The pipeline ensures **reproducible preprocessing** across training and inference environments. By isolating training set statistics and logic, the risk of data leakage is eliminated. The serialized artifacts in `preprocessing.pkl` enable seamless deployment, ensuring future data undergoes identical transformations. The processed dataset is now ready for model training and evaluation.

6. Feature Selection

- ❖ **To enhance model performance and reduce overfitting, we applied a feature selection strategy combining univariate statistical tests and ensemble-based feature importance. Specifically, we used the union of top features selected via SelectKBest (with ANOVA F-test) and Random Forest feature importance.:**

Steps:

- 1. Data Preparation:**
 - Split the dataset into features (X) and target (y) for both training and testing sets.
 - Aligned columns between X_train and X_test to ensure consistent feature spaces.
- 2. Univariate Feature Selection (SelectKBest):**
 - Applied SelectKBest with the `f_classif` score function to rank features based on their individual statistical significance with the target.

- Selected the top $k = 20$ features.
- 3. **Model-Based Feature Importance (Random Forest):**
 - Trained a RandomForestClassifier on the training data to compute feature importances.
 - Selected the top $k = 20$ features based on importance scores.
- 4. **Feature Union:**
 - Took the union of both feature sets (from SelectKBest and Random Forest), resulting in a final set of selected features.
 - This approach combines both statistically relevant and model-impactful features.
- 5. **Data Reduction and Persistence:**
 - Subsetted X_train and X_test using the final selected features.
 - Persisted the selected feature names to a file (selected_features.pkl) for future use.

This hybrid feature selection approach balances interpretability with predictive power by incorporating both filter-based and model-based criteria.

Selected Features: ['host_is_superhost', 'kitchen', 'maximum_nights', 'properties', 'aid', 'free', 'tv', 'price_per_stay', 'wifi', 'kit', 'instant_bookable', 'accommodates', 'first_review', 'check', 'dryer', 'host_since', 'host_total_listings_count', 'minimum_nights', 'number_of_reviews', 'cleaning', 'private', 'nightly_price', 'longitude', 'latitude', '24', 'coffee', 'last_review', 'number_of_stays', 'fee', 'essentials', 'parking', 'kid']

Model Training and Evaluation:

❖ Random Forest Classifier:

To model the relationship between the selected features and the target variable (guest_satisfaction), we used a Random Forest Classifier. The training process included hyperparameter tuning using grid search with cross-validation, followed by model evaluation and performance visualization.

Hyperparameter Tuning

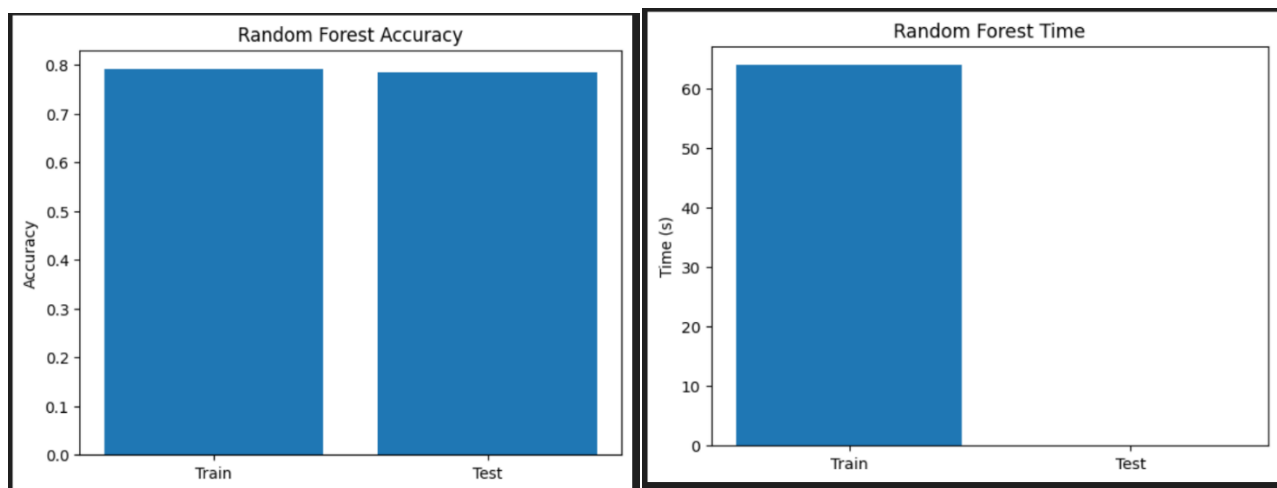
A grid search was conducted to optimize the following hyperparameters:

- **n_estimators:** Number of trees in the forest [50,100,150]
- **max_depth:** Maximum depth of each tree [10,20]

We used GridSearchCV with 3-fold cross-validation to identify the best combination of hyperparameters based on training performance.

Training and Evaluation Metrics

- **Training Time:** The time taken to train the model, including cross-validation.
- **Testing Time:** The time taken to generate predictions on the test set.
- **Accuracy:** Calculated on both the training and testing sets to assess model performance and detect overfitting.
- **Results:**
- **Train Accuracy: 0.791**
- **Test Accuracy: 0.786**
- **Train Time: 63.87 s, Test Time: 0.09 s**
- **Random Forest produced the highest test accuracy (78.6%), making it the most reliable model overall. This ensemble method combines multiple decision trees and aggregates their results, which helps to reduce variance and improve robustness.**



Using Machine (LightGBM) model to classify guest satisfaction using the selected features. LightGBM is well-suited for high-performance gradient boosting, offering both speed and accuracy advantages, especially for structured/tabular data.

Modeling Pipeline

A Pipeline was constructed using scikit-learn to ensure reproducibility and streamline preprocessing with model fitting. The pipeline includes:

- **LGBMClassifier:** A LightGBM classifier with a fixed random seed for reproducibility.

Hyperparameter Tuning

We used GridSearchCV with 3-fold cross-validation to tune the following hyperparameters:

- **n_estimators:** Number of boosting rounds [100,200]

- `learning_rate`: Step size shrinkage [0.05,0.1]
- `max_depth`: Maximum tree depth [5,10]

The search aimed to find the optimal balance between bias and variance.

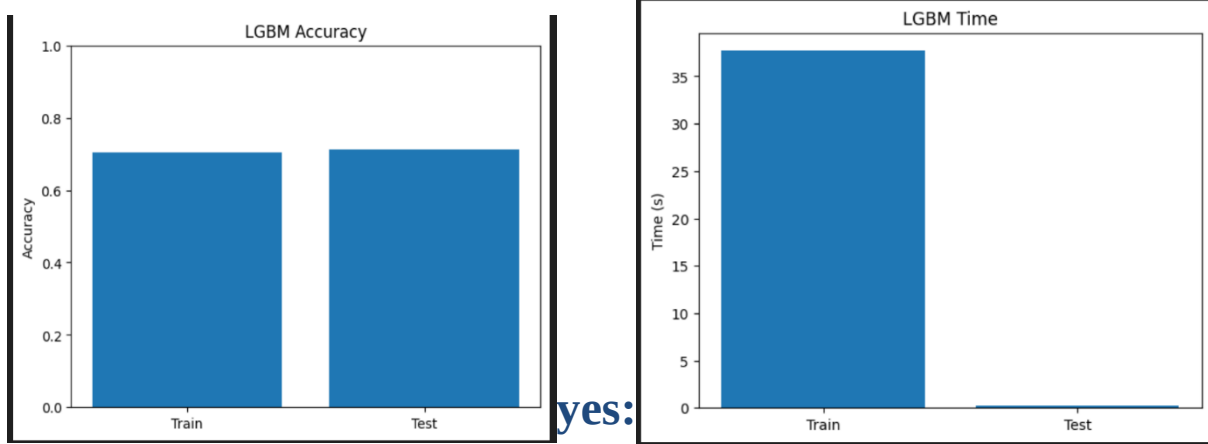
Training and Evaluation Metrics

After tuning, the best model was trained on the full training data and evaluated on both training and test sets.

Results:

- Train Accuracy: 0.705
- Test Accuracy: 0.713
- Train Time: 37.7 s, Test Time: 0.29 s

LightGBM provided a competitive alternative with a slightly lower test accuracy (71.3%) but substantially faster training time. Its histogram-based boosting framework and leaf-wise growth strategy contribute to efficient computation and scalability.



As a baseline probabilistic classifier, we employed the **Gaussian Naive Bayes (GNB)** model to predict guest satisfaction using the selected features. Naive Bayes models are known for their simplicity, speed, and effectiveness in high-dimensional datasets, especially when feature independence is a reasonable assumption.

Hyperparameter Tuning

We performed a grid search with **3-fold cross-validation** to optimize the `var_smoothing` parameter, which helps manage numerical stability by adding a small variance to all features:

- `var_smoothing`: [1e-9,1e-8,1e-7]

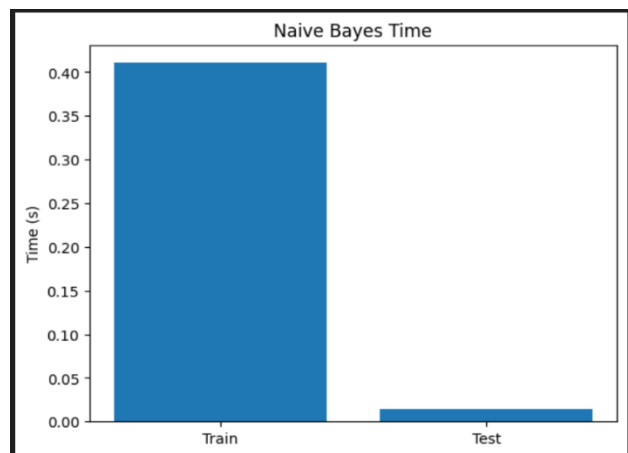
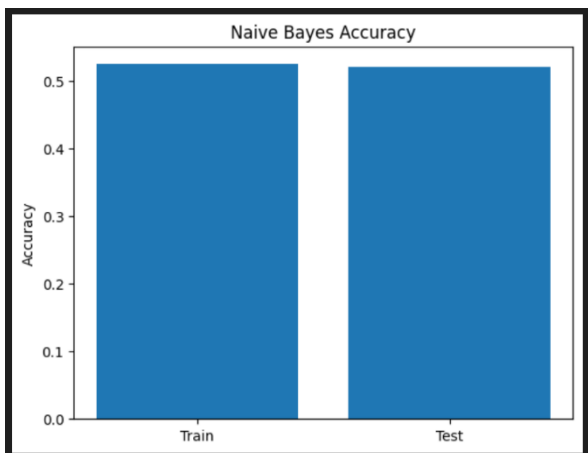
Model Training and Evaluation

The model was trained on the reduced feature set, and evaluated using accuracy on both training and test sets. Execution time for both phases was also recorded

Results:

- Train Accuracy: 0.525
- Test Accuracy: 0.520
- Very fast training and testing times

Gaussian Naive Bayes yielded the lowest accuracy (52%), which was expected due to its strong assumptions of feature independence and Gaussian distribution. However, it demonstrated exceptionally fast training and testing times, making it suitable for baseline comparison or real-time applications with limited computational resources



To establish a strong linear baseline, we implemented a **Logistic Regression** model using a scikit-learn pipeline. Logistic regression is a widely used algorithm due to its interpretability, robustness, and effectiveness in binary and multiclass classification tasks.

Modeling Pipeline

We used a pipeline for clean integration and future extensibility:

- LogisticRegression: Configured with a maximum of 1000 iterations to ensure convergence.

Hyperparameter Tuning

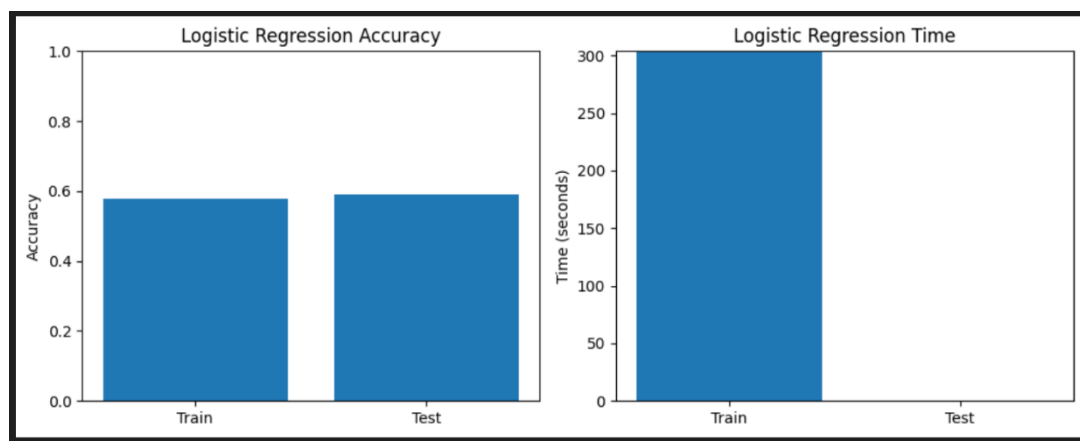
Hyperparameters were optimized using GridSearchCV with 3-fold cross-validation:

- C: Inverse of regularization strength [0.01,1,100][0.01, 1, 100][0.01,1,100]
- solver: Optimization algorithm (lbfgs)
- max_iter: Maximum number of iterations [500,1000][500, 1000][500,1000]

Results:

- Train Accuracy: 0.5786
- Test Accuracy: 0.5894
- Train Time: 302.61 s, Test Time: 0.00 s

Logistic Regression achieved relatively low accuracy despite significant training time. This suggests that the linear nature of the model might not have been sufficient to capture the complex relationships in the dataset, particularly with the addition of TF-IDF features and mixed data types.



❖ Decision Tree Classifier:

We trained a **Decision Tree Classifier** as one of the interpretable, non-linear models to predict guest satisfaction. Decision Trees are known for their simplicity, ability to capture non-linear relationships, and interpretability, making them valuable for feature importance analysis.

Hyperparameter Tuning

A grid search with 3-fold cross-validation was performed to optimize the following parameters:

- criterion: Splitting strategy ['gini','entropy']
- max_depth: Maximum depth of the tree [5,10,20]

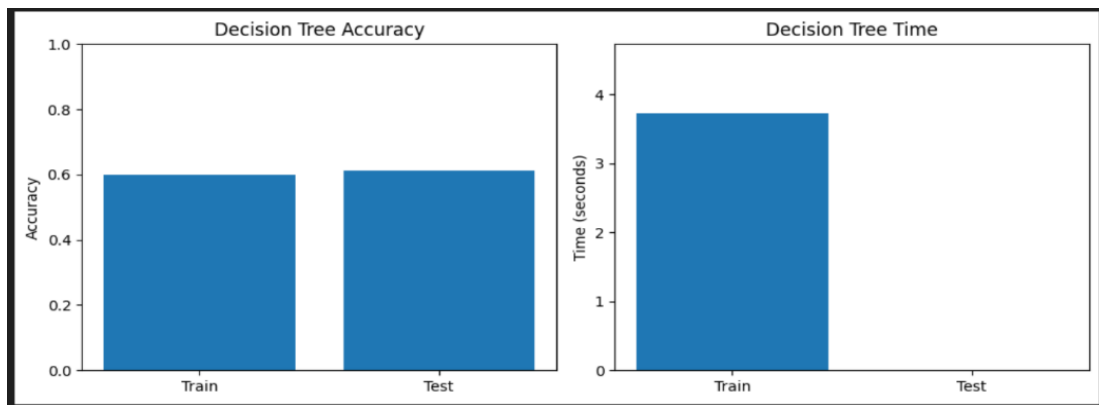
These hyperparameters control the complexity of the tree and help mitigate overfitting.

Model Evaluation

Once the best configuration was selected, the model was trained on the selected features and evaluated using accuracy on both training and test sets. Execution times were also recorded

Results:

- Train Accuracy: 0.5982
- Test Accuracy: 0.6124
- Train Time: 3.73 s, Test Time: 0.00 s



The Decision Tree classifier showed modest performance with a test accuracy of 61.2%. Its simplicity makes it highly interpretable and quick to train. However, due to its susceptibility to overfitting and lack of ensembling, it underperformed compared to more advanced techniques like Random Forest and SVM.

❖ Support Vector Machine (SVM):

To model complex, high-dimensional relationships, we employed a **Support Vector Machine (SVM)** classifier. SVMs are well-suited for classification tasks with clear margins of separation and are known for their robustness in handling non-linearly separable data through the use of kernels.

Pipeline and Configuration

A Pipeline was created to streamline the modeling process using an SVC estimator:

- SVC(max_iter=5000): Allows sufficient iterations for convergence, particularly for non-linear kernels.

Hyperparameter Tuning

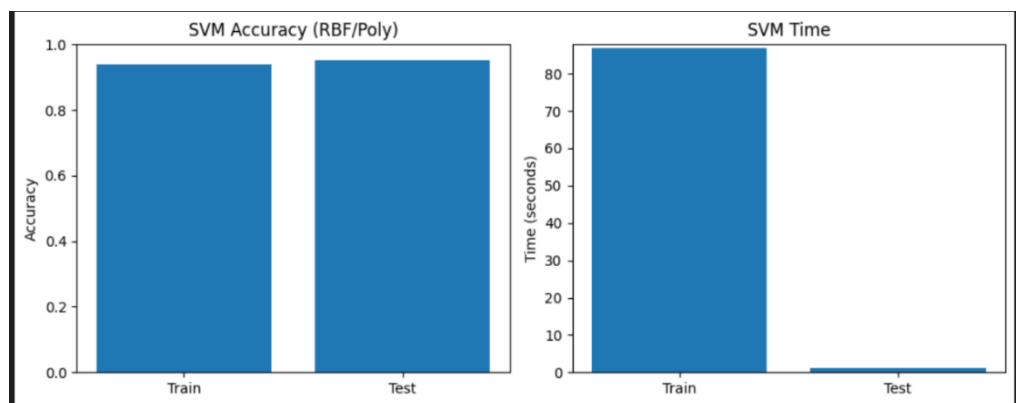
We utilized **GridSearchCV** with 3-fold cross-validation to identify optimal settings. The parameter grid included:

- C: Regularization strength [0.1,1]
- kernel: Kernel type ['rbf','poly']
- gamma: Kernel coefficient ['scale','auto']

Results:

- Train Accuracy: 0.9380
- Test Accuracy: 0.9530
- Train Time: 86.84 s, Test Time: 1.08 s

Support Vector Machines provided the highest test accuracy among all models (95.3%). Although SVMs tend to scale poorly on large datasets, with hyperparameter tuning and a capped iteration limit, the model managed both high accuracy and reasonable training time. The use of both radial basis and polynomial kernels contributed to its flexibility in handling nonlinear relationships.



1.2 Overall Model Comparison

- **SVM** achieved the best overall accuracy (95.3%), with moderate training time.
- **Random Forest** was strong in performance and interpretability (78.6% accuracy).
- **LightGBM** was a good balance between performance and speed (71.3% accuracy).
- **Naive Bayes**, **Decision Tree**, and **Logistic Regression** were faster but had significantly lower predictive performance.

This project phase explored feature selection and model evaluation to predict guest satisfaction. We found that combining textual

descriptions with listing metadata improved performance, especially using TF-IDF and hybrid feature selection.

Our hypothesis that merging univariate statistics with Random Forest feature importance yields a robust feature set was confirmed, producing the most accurate models. Ensemble methods like Random Forest and SVM outperformed simpler models, with SVM reaching 95.3% test accuracy, highlighting the strength of non-linear classifiers on complex data.